

Master's thesis

Master's Programme in Computer Science

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien

November 10, 2026

Faculty of Science

University of Helsinki

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki,Finland

Email address: info@cs.helsinki.fi
URL: http://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Study programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ohjaajat — Handledare — Supervisors

Prof. Keijo Heljanko, Christoph Lüneburg

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidoantal — Number of pages
Master's thesis	November 10, 2026	64 pages, 9 appendix pages

Tiivistelmä — Referat — Abstract

Radio access network (RAN) control planes are written mostly in C++ for performance, at a well-known cost in memory safety and potential for concurrency bugs. Rust promises comparable performance with compile-time safety, but its fit for this latency-critical setting is unproven.

The thesis compares modern C++ and Rust using a user equipment (UE) Context Management service that keeps per-device state, as a 5G base-station control plane (gNB-CU-CP) does. Average-throughput comparisons leave four dimensions open: tail latency, the defect classes Rust prevents at compile time, safety-model overhead, and incremental adoption.

One language-agnostic specification is fixed and then implemented in both C++ and Rust. The two versions use the same algorithms, thread layout, memory allocator, and build settings. Where C++ and Rust cannot match line for line, the difference is written down and kept in the measurement. Both versions pass the same parity tests. The benchmark then raises the request rate step by step until the service overloads. The evaluation also adds fault injection and a foreign function interface (FFI) prototype.

Under these experimental conditions, Safe Rust rejects all seven injected memory-safety and data-race defects at compile time, whereas the corresponding C++ implementation compiles successfully. However, both languages continue to compile programs containing other classes of defects, including deadlocks.

While the service can still keep up under normal load, a typical request is up to 1.5 μs faster in Rust, and Rust uses slightly less processor time. As the request rate increases and the service becomes overloaded, the result depends on the test clock. If the test writes its extra delay records while the clock is running, C++ finishes more requests. If those records are written afterwards, Rust finishes more, at 784 thousand requests per second with both memory libraries. Most of the lead is in the queue between the threads. When both languages use the same queue, Rust finishes 0.6% to 1.3% more requests with the operating system's memory library. When several requests wait at once, Rust waits less with `jemalloc`, and C++ waits less with the system's own library. On all three computers tested, Rust uses a few megabytes more memory. Existing C++ code can call the unchanged Rust service through a thin C interface. That extra step adds at most 10.91% of the time of one signalling message.

ACM Computing Classification System (CCS)
Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Avainsanat — Nyckelord — Keywords

Rust, C++, RAN control plane, tail latency, memory safety, fault injection, incremental adoption

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — övriga uppgifter — Additional information

Software study track

Contents

List of Figures iii

List of Tables iv

List of acronyms v

Key terms vi

1 Introduction 1

1.1 Motivation . . . 2
1.2 Industrial context . . . 3
1.3 Problem statement . . . 5
1.4 Research questions and hypotheses . . . 6
1.5 Contributions and outline . . . 7

2 Background: Systems Languages for Latency-Critical Software 8

2.1 RAN control-plane systems and the gNB-CU-CP . . . 8
2.2 Memory safety . . . 9
2.3 Research gap . . . 10

3 Research Methodology 13

3.1 The UE Context Management subsystem . . . 13
3.2 The defect catalogue . . . 14
3.3 Research approach . . . 15
3.4 Methods per research question . . . 17
3.5 Fairness as a controlled variable . . . 19
3.6 Statistical treatment . . . 19
3.7 Reproducibility and research artefacts . . . 21

4 Design and Implementation 22

5 Experimental Setup and Evaluation 25

5.1 Testbed setup . . . 25
5.2 Workload and measurements . . . 27
5.3 Results . . . 29
5.3.1 Below saturation (RQ1, RQ3) . . . 29
5.3.2 Past saturation (RQ1) . . . 33
5.3.3 Resource use (RQ3) . . . 38
5.3.4 Safety via fault injection (RQ2) . . . 40
5.3.5 Migration (RQ4) . . . 43

6 Discussion **44**
6.1 Summary of findings . . . 44
6.2 Discussion on the research questions . . . 45
6.3 Revisiting the hypotheses . . . 48
6.4 Engineering guidelines for incremental adoption . . . 49
6.5 Where this study sits . . . 50
6.6 Ethical and professional reflection . . . 51

7 Conclusion and Future Work **52**

Statement on the Use of AI Tools **55**

Bibliography **56**

A Host and campaign configuration

B Overload probes in detail

List of Figures

2.1 Where the studied subsystem sits in the 5G NG-RAN 9
2.2 A checklist of earlier comparisons 11

3.1 UE-context lifecycle state machine 13

4.1 Shared-nothing, thread-per-shard architecture 22
4.2 The request type in each language 23

5.1 The two test workloads 27
5.2 The load sweep: tail latency and achieved throughput 31
5.3 Where the end-to-end clock goes 32
5.4 How small the language's share of a round-trip is 33
5.5 Four load points, from a light load to overload 35
5.6 One handoff written the same way in both languages 37
5.7 Peak memory and CPU of the four builds 40
5.8 Use-after-move in both languages 41
5.9 Where the guarantee stops: the outcome matrix 42
5.10 The thin layer where C++ calls the Rust service 43

6.1 Where Rust's compile-time guarantee stops 47

List of Tables

3.1 Injected defects and the expected outcome 15
3.2 How each research question is answered 17

4.1 Where the two languages had to differ 23

5.1 Delay of one call at the chosen load 30
5.2 The four planned comparisons past saturation 34
5.3 Follow-up runs past saturation 36
5.4 Time spent in each part of one call 37
5.5 Delay with several requests in flight 38
5.6 Extra peak memory on each host 38

A.1 The three machines ii
A.2 Settings of the service, the workload, and the campaign iii
A.3 The ten planned comparisons below saturation iv
A.4 How much the delay changes from one call to the next vi
A.5 Where the extra memory sits vii

B.1 The other machines used for the follow-up runs i

List of acronyms

Each acronym below is also written out where the text first uses it. Terms that are not acronyms, such as *tail latency* and *shard*, are collected in the list of key terms that follows.

Acronym	Expansion
3GPP	3rd Generation Partnership Project, the body that specifies the 5G standards
ABI	Application binary interface
AI	Artificial intelligence
AI-RAN	An architecture in which AI and radio access network workloads share one accelerated infrastructure
API	Application programming interface
ASN.1	Abstract Syntax Notation One
CPU	Central processing unit
CVE	Common Vulnerabilities and Exposures
DPDK	Data Plane Development Kit
EU	European Union
FFI	Foreign function interface
FIFO	First in, first out
FNV-1a	Fowler–Noll–Vo hash function, variant 1a
gNB	Next-generation NodeB, the 5G base station
gNB-CU	gNB central unit
gNB-CU-CP	gNB central unit control plane, the domain this work sits in
gNB-CU-UP	gNB central unit user plane
gNB-DU	gNB distributed unit
HDR	High dynamic range, of the latency histogram
IMT-2030	International Mobile Telecommunications-2030, the ITU framework for 6G
ITU	International Telecommunication Union
KPI	Key performance indicator
kops/s	Kilo-operations per second
KVM	Kernel-based Virtual Machine
MPSC	Multi-producer, single-consumer
NG-RAN	Next-generation radio access network
PHY	Physical layer
RAII	Resource Acquisition Is Initialization
RAN	Radio access network
RQ	Research question
RRC	Radio Resource Control
RSS	Resident set size
UE	User equipment, an end device such as a phone or a modem
UECM	UE Context Management, the subsystem reimplemented and measured here

Key terms

The table below gives plain meanings for the network, the service, the timing, and the safety words used in the thesis.

Term	Meaning
Network	
Control plane	Messages used to set up, manage, and end a call. User data itself travels on the user plane.
User plane	The data path that carries the user's traffic. The control plane creates and removes this path.
Signalling	Messages used to create, modify, or end a connection.
Handover	Moving a connected device from one cell to another so the call can continue.
Cell	One radio coverage area in the network.
Mobility	A device changing cells while its connection continues.
UE context	The stored network record for one device. It can change during actions such as handover or release.
Bearer	A bearer is a data path used to carry one device's user traffic, and the UE context keeps a record of the bearers assigned to that device.
Reconfiguration	A change to an active connection, such as adding a data channel.
Release	Ending one device's connection and deleting its record.
Procedure	A complete network action, such as setting up a call or moving between cells. It consists of several smaller steps.
Open-RAN	A base station design with open interfaces between its parts. Different parts can come from different vendors and run as software.
5G core	The network behind the base station that routes and handles the call or data session.
The service	
Shard	One part of the device records, owned by exactly one thread.
Producer	A thread that sends test requests and waits for replies.
Worker	A thread that owns one shard and processes requests for it.
Bounded queue	A fixed-size queue in front of a shard. If it becomes full, requests wait instead of being dropped.
Operation	One request for one device, such as setup, reconfiguration, or release.
Variant	One of the four tested builds: C++ or Rust, each combined with one of two memory allocators.
Allocator	The memory library that gives memory to the program and takes it back when it is no longer needed.
Timing	
Tail latency	How long the slow calls take, rather than the average call. A p99 latency means 99 of 100 calls were faster than that time, a p99.9 means 999 of 1,000, and a p99.99 means 9,999 of 10,000.

Term	Meaning
Median	The middle call, once the calls are ordered by how long they took. Half are faster and half are slower.
Histogram	A record showing how long calls took. Tail latency is the slowest part of that record, showing the longest call times.
Send schedule	The planned send time for each request. If a producer is late, it sends the next request immediately after the reply.
Offered load	The rate the test tries to send, counted whether or not the service answers in time.
Achieved rate	How fast the service finishes work. It matches offered load until saturation.
Throughput	How many requests the service finishes each second.
Saturation	The load at which the service can no longer keep up and requests start to wait.
Overload	Load past saturation. Requests wait because the service cannot keep up.
Round-trip	The time from the planned send time until the reply is received, including any queue waiting time.
Bare metal	Running directly on the computer hardware, without a virtual machine.
Campaign	One complete timing experiment on one machine. It includes repeated runs of the same loads and the same four builds.
Hypervisor	Software that runs virtual machines. The reported bare-metal campaign does not use one.
Guest	A virtual machine used in an earlier measurement. The reported results come from the bare-metal machine.
Userland	Programs and libraries running above the kernel. The container provides the compilers, while the host provides the kernel.
Safety	
Data race	Two threads access the same memory at the same time, at least one writes, and nothing controls the order.
Race condition	A bug where the result depends on which of two steps happens first. Safe Rust can still have race conditions.
Use-after-free	Using memory after it has already been freed.
Double-free	Freeing the same memory twice.
Use-after-move	Reading a variable after its contents have been moved elsewhere. The object is still there and still valid, but what it holds is unspecified.
Ownership	Rust's rule that each value has one owner. The compiler rejects use of a value after its owner is gone.
Borrowing	Temporarily using a value without taking ownership. The compiler rejects borrows that last too long.
Safe Rust	Code outside `unsafe` blocks, where ownership, borrowing, and aliasing are checked at compile time.
Unsafe	An `unsafe` block or function where the programmer takes responsibility and the usual checks do not apply. C++ has no such mark.
Sanitiser	A tool that checks a running C++ program and reports some memory or safety errors. It only checks code paths that actually run.
Undefined behaviour	An operation the C++ standard gives no meaning, such as signed integer overflow. The compiler may assume it never happens.
Static analyser	A tool that checks source code without running it. This study did not use one.

1 Introduction

Mobile networks are moving from specialised hardware to software on ordinary servers. Functions that once ran on vertically integrated equipment, built for one job and sold as one piece, are now reimplemented as programs on general-purpose machines. The Radio Access Network (RAN) connects to mobile phones wirelessly over the air. Traditionally, it has been one of the most hardware-dependent parts of the network stack, relying heavily on specialized equipment. However, it is now undergoing the same shift toward more flexible, software-driven infrastructure.

In 5G, the base station is the radio site a phone connects to. The standards call it a next-generation NodeB (gNB). They split it into a central unit and one or more distributed units, and they split the central unit again into a control plane and a user plane [1, 2]. The control plane carries the signalling, the messages that set up and control the call. The user plane carries the phone's data. Open-RAN pushes that split idea further: the RAN becomes virtualised, multi-vendor software linked by open interfaces, so each part may come from a different company [3]. The control plane that results, the gNB central unit control plane (gNB-CU-CP), tracks every connected device, which are called user equipment (UE). It keeps a record for each of them and handles many at the same time. A program that performs both functions is considered stateful and highly concurrent. The control plane runs Radio Resource Control (RRC) signalling [4] and coordinates mobility and handovers, moving a live call from one coverage area to another, all within strict latency limits.

A single site runs many connection-management and mobility procedures every second, such as setting up a call or moving it between cells. Each procedure has to complete within the millisecond-scale budget the 5G system sets for control-plane signalling [5]. A deployed control plane tracks every connected device at once. The RAN is moving to cloud-native deployments, onto ordinary servers instead of purpose-built hardware. It is also moving to disaggregated ones, where the parts may come from different companies. The correctness, reliability, and timing of this software then increasingly set the quality of the whole network. An outage here is not a local defect. Neither is a tail-latency spike: a few calls take much longer than the rest, even when the average still looks fine. Every device the site serves is affected [6, 7].

This control-plane software is, today, written mostly in C++. The choice is a practical one: C++ offers the low-level control, mature tooling, and performance that latency-critical

systems demand. But that performance comes with a well-known cost. C++ places the burden of memory and concurrency management on the programmer, and decades of experience show that this burden is routinely mishandled. The typical failures are use-after-free errors, double-frees, data races, and related defects. They are expensive to find, dangerous in production, and responsible for a large share of security vulnerabilities in industry practice [8, 9]. Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile time on machine-checked foundations [10, 11], with performance intended to be competitive with C++ [12]. Whether that promise holds for a telecom-grade, latency-critical control plane is not well understood. Predictability under overload matters as much as raw speed there. This thesis, carried out at Nokia, addresses the question directly. It develops two functionally equivalent implementations of a representative RAN subsystem, a UE Context Management service that keeps per-device state, as a gNB-CU-CP does, in modern C++ and in Rust. It compares them under a realistic synthetic signalling workload.

Three terms recur throughout, and the first is the *gNB-CU-CP*, the control-plane class this work sits in. *UE Context Management (UECM)* is the compact service reimplemented and measured. *Tail latency* is the third. More than average speed, it decides whether such a system meets its latency budgets [6, 7].

1.1 Motivation

Three trends make this comparison timely.

- **Cloud-native RAN.** Control-plane functions now run on general-purpose servers, so the software is longer-lived, more concurrent, and must meet its latency budgets without the determinism of purpose-built hardware. Open-RAN disaggregation turns the base station into virtualised, multi-vendor software [3]. A latency regression or a memory-corruption defect is then a network-wide risk, and the language choice is an engineering decision.
- **AI on the RAN host.** Artificial intelligence (AI) used to be only a tool. It helps to choose radio settings, such as how much power to send. That tool does not have to run on the radio's own server. Now AI programs also run on that server, next to the radio software [13]. Both run at the same time, so they share one processor. International Mobile Telecommunications-2030 (IMT-2030) is the 6G framework of the International Telecommunication Union (ITU). It lists AI together with communication as one

named job for 6G [14]. The architecture forming around that job is AI-RAN, in which AI and radio work share one accelerated server [15]. A control plane there no longer has the host to itself. It shares the central processing unit (CPU), memory, and cache with inference work it does not schedule. Inference here means running a trained model so that it produces an answer. When that neighbour takes capacity, the control plane saturates sooner: it stops keeping up with demand at a lower load than it would alone. Chapter 5 therefore drives both implementations past the saturation point. It also separates the time spent passing work between threads from the time spent doing the work itself.

- **Memory-safe languages.** Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages reduces them [8, 9, 16, 17]. Independent work argues that memory safety now needs standardised requirements if it is to be procured at all [18]. Cybersecurity agencies urge a move away from C and C++, and the European Union (EU) Cyber Resilience Act adds legal duties from September 2026 and December 2027 [19, 20, 21]. The Act names no programming language. Direct evidence for a RAN control plane is still missing. Providing it is the motivation for this work.

1.2 Industrial context

This thesis is carried out at Nokia, one of the few vendors that develop, maintain, and evolve production Radio Access Network (RAN) software end to end. The RAN vendor market is highly concentrated: the five largest suppliers accounted for 96% of worldwide RAN revenue during the first three quarters of 2025, while the Herfindahl–Hirschman Index, a measure of market concentration, reached a ten-year high [22, 23]. This context makes production RAN software an important setting in which to study software design choices.

The focus of this thesis is UECM, a subsystem that provides a compact representation of the per-user-equipment, or per-UE, state management performed by the gNB-CU-CP. In the 3rd Generation Partnership Project (3GPP) architecture, the gNB-CU-CP is the control-plane part of the central unit. It hosts the Radio Resource Control (RRC) function and manages the lifecycle of UE contexts [1, 4]. These tasks must be performed within the millisecond-scale timing constraints defined for 5G control signalling [5]. Chapter 2 explains how this control plane fits into the broader design of the 5G base station, the gNB.

UECM is particularly suitable for this thesis because it is small enough to reimplement faithfully and analyse in detail, while still capturing the challenges that make control-plane software difficult. It is stateful, highly concurrent, and performance-sensitive. Since it is implemented in C++, it also reflects the trade-offs of that language: high performance on the one hand, and memory-safety and concurrency risks on the other. In this way, UECM brings together the three trends introduced in Section 1.1. Because software of this kind runs in live operator networks, the choice of programming language is not only a technical detail but also a production concern.

The implementation used in this thesis follows a public, fixed specification rather than proprietary Nokia code. This allows readers outside Nokia to reproduce the comparison and assess the results independently.

A decision with a generational horizon. A language choice in software of this class outlives the generation it was made for. 3GPP has allocated Release 20 to the 6G studies and Release 21 to the normative work. The ITU asks for technology proposals in early 2029 and for specification submissions by mid-2030 [24]. This work includes migration from 5G and interworking with existing 5G systems. As a result, control-plane software developed today is likely to be maintained alongside the next generation rather than replaced immediately. IMT-2030 also continues to identify security and resilience as core design principles, while retaining and strengthening the ultra-reliable low-latency communication scenario [14].

The measurements here use a current-generation subsystem and published requirements.

A concrete failure mode. A moving train full of passengers stands for the connected UEs at one site. A handover moves one passenger from one cell to another, and the call can continue. Each passenger has a stored record, the UE context. When many passengers move at once, the control plane is under a mobility storm. A handover for one passenger is still in progress. The cell the passenger is leaving deletes the stored record, while a message for that passenger is still being handled. Which of those two steps finishes first is a race condition. If the read comes after the delete, it is a use-after-free which still compiles in C++. Tests that do not hit that ordering can pass. When many calls run at once, the bug can corrupt another passenger's record. That race can crash the thread that is serving the call. A few calls can become much slower than the rest. That jump in the slowest calls is a tail-latency spike. In safe Rust the same design does not compile: ownership and borrowing reject the read of a record that has already been deleted. That is why this thesis measures both *safety* (which defects are prevented, and when) and

predictability (tail latency under overload), not average throughput alone.

1.3 Problem statement

Two properties of this domain make the language choice consequential. First, manual memory and concurrency management in C++ exposes the software to defects such as use-after-free, double-free, data races, and iterator invalidation. These defects are silent at compile time and costly to find and fix at telecom scale. Second, what governs whether RAN latency budgets are met is not average throughput but worst-case behaviour and tail latency under load and overload [6], a property that is easy to regress and hard to measure correctly. Rust promises to remove the first class of defect at compile time while claiming competitive performance on the second, but that claim rests mostly on general-purpose benchmarks rather than on stateful, concurrent, latency-critical telecom workloads.

Bringing these threads together, the concrete problem this thesis addresses is the following. Telecom vendors are under growing pressure, both technical and increasingly regulatory, to adopt memory-safe languages. For the RAN control plane they still lack *direct, controlled* evidence on which to base that decision. It is not established whether a Rust implementation of a stateful, concurrent control-plane subsystem can match modern C++ in *tail latency and predictability under overload* (as opposed to average throughput). It is also unknown whether Rust's compile-time guarantees actually eliminate the specific defect classes that affect the existing C++ code, and what incremental adoption *costs* inside a large C++ codebase. The available evidence is largely general-purpose and average-case [12, 25]. For a representative subsystem, four questions therefore stay open:

- **Predictability:** tail latency and behaviour under overload, not average throughput alone.
- **Prevented-defect scope:** which C++ defect classes Rust removes at compile time, and how the rest are caught.
- **Resource overhead:** throughput, CPU use, and memory under the safety model.
- **Migration cost:** effort, foreign-function boundaries, and risk inside a large C++ codebase.

Answering them under controlled, reproducible conditions is the problem this thesis sets out to solve.

1.4 Research questions and hypotheses

The thesis is organised around four research questions. They put latency predictability first, then defect scope, then resource cost, then incremental adoption:

RQ1 How do Rust and modern C++ compare in latency predictability (median, p99 latency, p99.9 and p99.99 tail latency, and the maximum latency, the slowest call in a run) for a RAN control-plane workload under increasing load and overload?

RQ2 Which classes of memory-safety and concurrency defects that occur in C++ are eliminated at compile time in the Rust implementation, and at what stage and cost are the remaining ones detected?

RQ3 What is the overhead, if any, of Rust's safety model (bounds checks, ownership-driven copies, abstraction cost) on throughput, CPU utilisation, and memory behaviour?

RQ4 What are the practical implications (effort, foreign function interface (FFI) interoperability, and risk) of adopting Rust incrementally within an existing C++ RAN software ecosystem?

Three hypotheses are written down before any measurement.

H1 For this workload the two languages are performance-equivalent below saturation, with any difference only among the slowest calls.

H2 Safe Rust prevents at compile time the memory-safety and concurrency defect classes that C++ only tries to catch after the program is built.

H3 Calling the Rust service from C++ through the FFI adds only a small share of one signalling call's time. The real cost of adding Rust one piece at a time is engineering work, not slower calls.

Chapter 6 revisits all three hypotheses against the facts.

The compile-time safety claims in RQ2 apply only to *safe Rust*, meaning code outside `unsafe` blocks, where the compiler can enforce Rust's safety rules (Section 2.2).

The RQ2 list has fourteen defect classes. Safe Rust is expected to stop seven of them at compile time. It is expected not to stop the other seven. That split is the boundary the study looks for, not only a check that the guarantee works. The seven boundary classes are these:

- lock-order deadlock

- lost update
- reference-cycle leak
- integer overflow in a release build
- worker-failure blast radius
- double-free inside `unsafe`
- forgotten lock guard

1.5 Contributions and outline

The thesis leaves three artefacts for reuse: a fixed, language-neutral specification of the service, a C++ and a Rust implementation that pass the same parity tests, and a timing harness that reaches past saturation. The first of its results compares latency and resource use below and past saturation, and separates the core operation from the handoff around it. The second maps where Rust's compile-time guarantee stops, using defects built on both sides of it. The third bounds what one call from C++ into Rust costs. The discussion turns these into guidelines for adding Rust one piece at a time.

Chapter 2 places the work in the 5G base station and among earlier comparisons, and introduces the two safety models and how tail latency is measured. Chapter 3 discusses the method for each research question. Chapter 4 describes the two implementations, and Chapter 5 the machines, the workload and the results. Chapter 6 answers the research questions, revisits the hypotheses and states the limitations, and Chapter 7 concludes.

2 Background: Systems Languages for Latency-Critical Software

2.1 RAN control-plane systems and the gNB-CU-CP

This thesis compares modern C++ and Rust in the next-generation radio access network (NG-RAN). In this network, user equipment (UE) moves between cells, generating signalling events such as handovers. These events can occur frequently and place a high load on the network. At the same time, signalling procedures must complete within strict time limits, as defined by the key performance indicators (KPIs) in the 5G service and RAN specifications [5, 26].

NG-RAN software is continuously developed and must handle many operations at the same time. This makes it important to understand not only how quickly operations usually complete, but also how slowly they can complete under heavy load. For example, a handover must finish within a specified time limit. If most handovers finish quickly but a few take too long, the average latency may still look good even though the time limit is exceeded for some handovers. These delays decrease the user experience and lead to KPI degradation

For this reason, this thesis focuses on tail latency, which describes the delays experienced by the slowest operations. By measuring tail latency, the comparison can reveal performance problems that average latency alone may hide.

The gNB is split in two ways. The first split is central versus distributed. The gNB runs a protocol stack: a set of layered radio protocols from the air interface up to RRC signalling. A central unit (gNB-CU) hosts the higher layers. One or more distributed units (gNB-DU) host the lower layers. Those layers sit close to the radio and have strict low-latency requirements. The second split is control versus user plane, inside the central unit. It splits the control-plane function (gNB-CU-CP) from the user-plane function (gNB-CU-UP). These parts communicate through standard links. The Uu interface is the radio link from the device to the distributed unit of the gNB. The NG interface runs between the gNB and the 5G core. F1 sits between the central unit and the distributed unit, and E1 sits between the control-plane half and the user-plane half [1, 2, 27]. Open-RAN pushes the same links further. Each part can then come from a different vendor and run as

software on ordinary machines [3]. Figure 2.1 shows the part studied in this thesis, UE Context Management (UECM), inside the gNB-CU-CP. UECM owns the per-UE control-plane state. It also handles RRC signalling and mobility. These are integral control-plane functions, which makes UECM a representative stateful subsystem. It does not stand for the control plane as a whole.

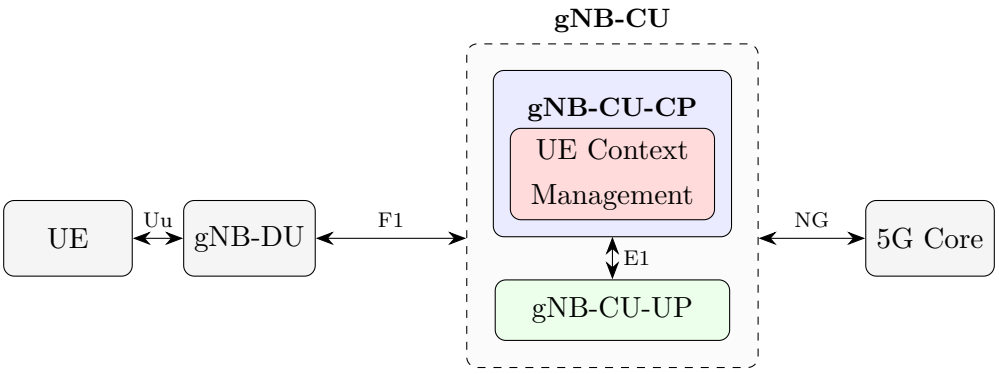


Figure 2.1: Where the studied subsystem sits in the 5G NG-RAN. UE Context Management is highlighted inside the gNB-CU-CP.

2.2 Memory safety

Rust separates code into *safe Rust* and code that uses `unsafe`. In safe Rust, the compiler checks how memory is used, including ownership and borrowing, to prevent many common programming errors [10, 11]. When code uses an `unsafe` block or an `unsafe` function, the programmer must make sure that certain operations are used correctly, because the compiler cannot check all their safety requirements. Unsafe code is often needed when Rust calls C code through the C application binary interface (ABI) or uses raw pointers, memory addresses the compiler does not check. C++ has no keyword that marks such code.

C++ manages many resources through Resource Acquisition Is Initialization (RAII), in which an object releases what it holds when it goes out of scope. Nothing in the language stops a program from keeping a pointer to that object afterwards, or from sharing it between threads without a lock. C++ projects therefore catch these mistakes with tools that watch a running program. This study runs three of them, AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer (Section 3.4). A sanitiser can report a defect only on a path a test actually runs, so a clean report covers the tests and not the program.

2.3 Research gap

The RAN control plane has many threads and tight latency bounds. Tail latency under too much load, not mean throughput, sets the quality the network delivers. C++ gives the control and speed the domain needs. But it defers detection of whole classes of memory and concurrency bugs to runtime. Rust promises to block those classes of errors at compile time at no run-time overhead. It also offers a step-by-step adoption path through the C application binary interface (ABI).

Five vendor-hosted Master's theses, all done at Ericsson, have compared candidate languages for RAN and telecom software. They do not all favour Rust. One compares run-time performance with development effort for C++, Rust, and Go [28]. One tests whether Rust can replace C in RAN node management [29]. One builds a packet processor in async Rust, with one thread per core, and measures it against the Data Plane Development Kit (DPDK) program it mirrors [30]. One measures median and p99 latency for control-plane steps across five async frameworks [31]. The last builds matched C++ and Rust versions of one program. That program turns a packed Abstract Syntax Notation One (ASN.1) message back into a readable form. It compares them with statistical tests [32]. All five are directly relevant. Olsson and Stenström's async Rust packet processor reaches a fifth to a half of the DPDK program's throughput, and its latency is far worse [30]. Arvidsson and Kaller's Rust p99 rises more steeply past a threshold rate [31]. Karlbäck and Orö pick C++ where speed is critical. They still find Rust lightest on peak memory [28]. Olsson's management software is as fast as C or faster. It uses more peak memory [29]. Emberg and Wadsten's Rust version is comparable to C++ in one session. In the next, its execution time is 85–91% longer than the C++ version [32].

Three of them change the language together with the run-time and the thread model, so a measured gap cannot solely be blamed on the language, but potentially on the framework as well [28, 30, 31]. In that packet-processor study the two programs differ in more than the language. The Rust program uses an async library to schedule its work, and the other program is built on DPDK. The DPDK program then finishes two to five times as much in the same time [30]. A difference in the library can be that large. A compiler translating the same design cannot change the result by that much. Olsson's management study and Emberg and Wadsten do not change the run-time with the language [29, 32]. They report execution time, and Olsson also reports peak memory. Neither measures tail latency under rising load. Emberg and Wadsten come closest in method to this thesis. Their program keeps no state between calls. The service in this thesis stores state and uses many threads. Figure 2.2 places the five vendor theses beside the general-purpose comparisons [33] and

the matched systems-software pairs [34, 35]. Each thesis groups leave out at least one aspect, that this thesis covers. No group keeps the design fixed, measures tail latency, and drives the service into overload.

All five studies come from one vendor. Their conclusions have never been tested against another firm's software, targets or house rules. So what is known about this question is what one firm's frameworks say about it. The published record on the other side is thinner. There is no study of Rust among the indexed papers of the vendor hosting this work. The nearest peer-reviewed stance from that side argues for making memory-error sanitisation practical. It does not argue for replacing it with a compile-time guarantee [36]. That is the strategy this thesis tests against.

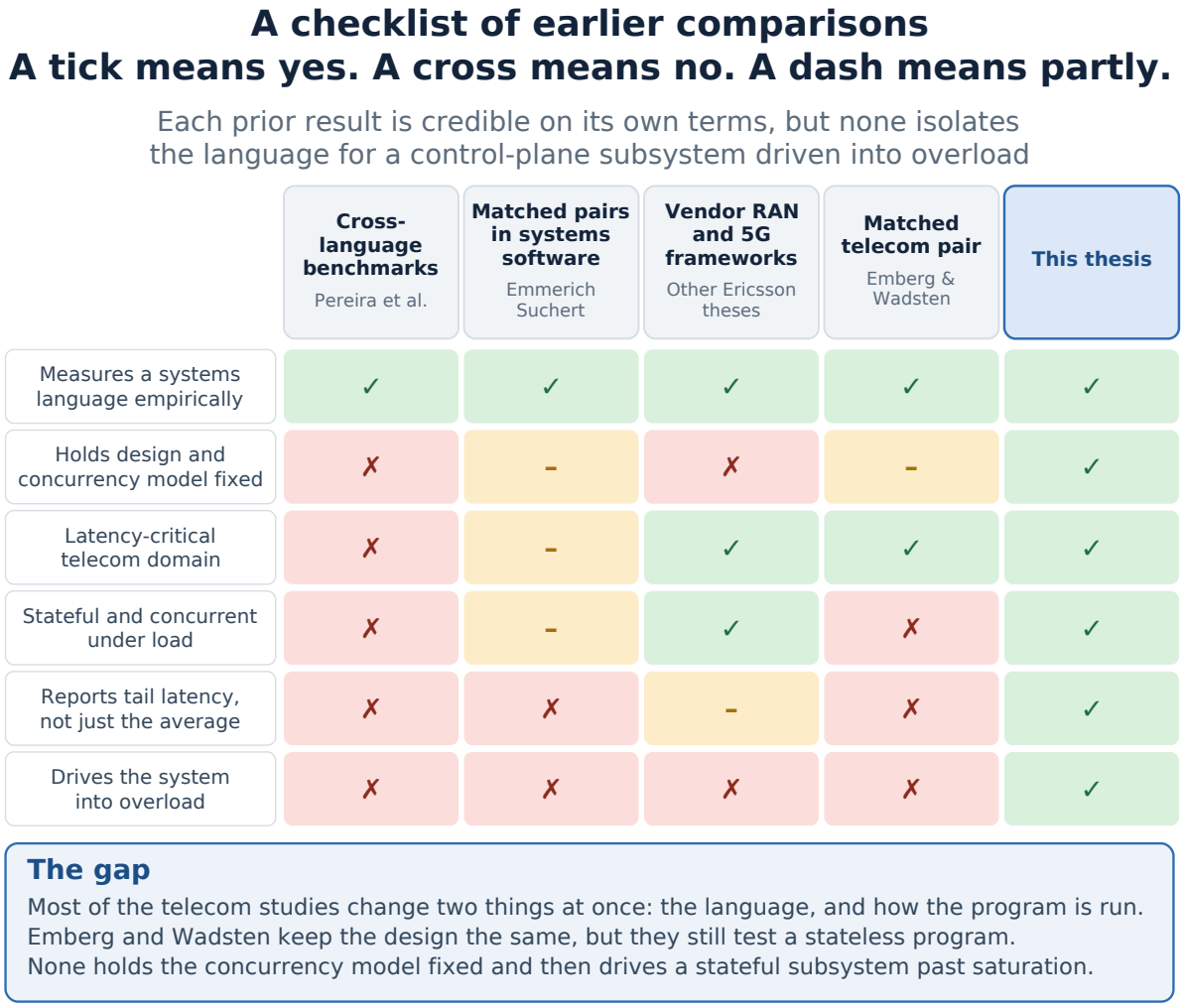


Figure 2.2: A checklist of earlier comparisons. Each row is a property a controlled answer would need, and each column is a group of studies. The vendor-RAN column is the other Ericsson theses. Emberg and Wadsten have their own column because they keep the design the same in both languages.

What is missing from prior work is a direct, controlled test of a typical control-plane part. The open questions are these. Does Rust hold its tail latency under too much load against modern C++ when the thread model, operations, and data layout stay fixed? Which bug classes does it remove, and at what stage are the rest caught? What does its safety model cost in throughput and in CPU and memory? What does step-by-step adoption cost in practice? Closing this gap is the purpose of the four research questions. The next chapter grounds them in the part under study, the bug classes, and the timing at stake. It also sets out the method.

3 Research Methodology

After explaining the research gap in the previous chapter, this chapter shows the methodology used in this thesis to close it.

3.1 The UE Context Management subsystem

The part under study is a UE Context Management service. Its operations follow the UE-context lifecycle, RRC procedure execution [4], and mobility and handover that the 3GPP design assigns to the control-plane part of a split gNB (Section 2.1). Figure 3.1 shows the UE-context lifecycle both versions encode the same way. The mixed workload of Section 5.2 walks each UE through that state machine.

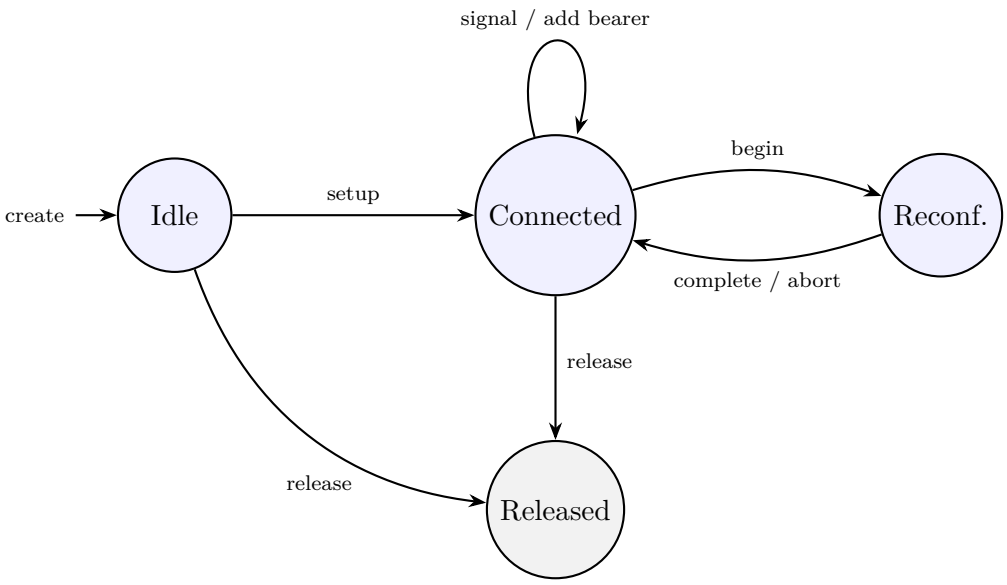


Figure 3.1: UE-context lifecycle state machine. Operations invalid for the current state return an error instead of changing state.

UECM is core RAN layer-3 work. It has the traits that make live control-plane code difficult. Each UE has a state that changes, the operation lifecycle is strict, and many UEs run at once. A delay target is the longest a control-plane procedure may take. The 5G service requirements name latency as one such limit [26]. For the move from a battery-saving idle state to the start of sending data, that limit is 10 ms [5]. The procedure is a chain of steps. It is late if the chain is late, even when most steps are fast. Findings about how a language handles those traits are most relevant to other stateful, multi-thread

control-plane services of this kind. The part is also small enough to specify in full and build in C++ and Rust. That is what makes a parity-tested comparison possible.

The interface is small and built around the lifecycle. It can create and release a context, set up a connection, handle an in-session signalling message, add a data radio bearer, begin, complete, or abort a reconfiguration, and look up a context's state under load. Each operation is defined for each state of Figure 3.1. An illegal operation returns a typed error and leaves the context unchanged. That closed set lets the same tests and the same workload drive both versions along the same paths.

As Section 1.4 states, RQ2 asks which defects safe Rust stops at compile time, and which ones it does not. Section 3.2 sets out the catalogue. For each defect, the expected result was written down before the programs were run. Section 5.3.4 reports how the coding languages handled the different defect classes.

The study reads tail latency and the delay of the middle request. Two aspects were taken into consideration when choosing to measure these. Firstly, all the processes needed for a phone user to set up a call or perform a handover to another cell. These are not one step processes, but a chain of steps happening in the control-plane, which have to finish in order. If all the steps happen in order, the user waits for all the steps (also slow ones) to be done. A busy site runs many of these chains every second. Even if only a small percentage of the chains are resolved slowly, due to the high amount of calls, the impact is still noticeable.

Secondly, the test clock is of interest, as each request is planned with a specific send time. If a sender falls behind it's schedule, it adds lag to the overall system. When the system stays below saturation, the achieve rate is solid and delays stay small.

3.2 The defect catalogue

The defects considered in this study fall into two broad families: *memory-safety defects* and *concurrency defects*. This classification is useful because the two families capture different ways in which C++ control-plane code can fail, and because safe Rust is expected to provide different guarantees for them.

Table 3.1 lists the 14 classes injected in Chapter 5, each in the form it would take in UE Context Management. The third column is how the defect fails if it occurs. The last column is what safe Rust was expected to do, written down before the study was run. Section 5.3.4 and Figure 5.9 report what was measured.

Table 3.1: Defects the study injected, and how safe Rust was expected to handle these. The third column is how the defect fails if it occurs. Buffer overflow is listed but not injected.

Defect	Example in UECM	How it fails	Safe Rust
Memory safety			
Use-after-free	Handler uses a context after it was released	Crash or corruption	Prevented
Double-free	Release frees the same context twice	Crash or corruption	Prevented
Use-after-move	Request uses a context after it was moved	Empty identity	Prevented
Buffer overflow	No applicable scenario in the context store	n/a	Not injected
Concurrency			
Iterator invalidation	Bearer list changes while it is being iterated	Crash or corruption	Prevented
Data race	Two workers update the same field at once	Wrong value	Prevented
Unguarded shared mutation	Multiple workers mutate the same context	Wrong value	Prevented
Thread-unsafe handle	Non-atomic handle is shared between workers	Corrupted refcount	Prevented
Defects safe Rust does not automatically prevent			
Lock-order deadlock	Two handovers acquire locks in opposite orders	Hang	Not prevented
Lost update	Two workers overwrite each other's updates	Wrong value	Not prevented
Reference-cycle leak	Contexts keep references to each other	Memory leak	Not prevented
Integer overflow (release build)	A bearer's tunnel identifier overflows a signed 32-bit integer	Wrong value	Not prevented
Worker-failure blast radius	Worker fails while holding shard state	Wider outage	Not prevented
Double-free inside `unsafe`	C++ releases a Rust handle twice	Crash or corruption	Not prevented
Forgotten lock guard	Lock stays held after an early return in C++, or a leaked guard in Rust	Hang	Not prevented

3.3 Research approach

The study runs experiments on a matched pair of the UE Context Management service. One is written in C++ and the other in Rust. It builds both, plus the harness that drives them. The harness is the program that sends the requests. It records how long each request takes. That time starts when the request should have been sent, and stops

when the reply comes back. Both versions are built from one fixed specification. Evidence comes from three separate methods: a load sweep contributes to RQ1 and RQ3, controlled fault injection answers RQ2, and an FFI migration prototype answers RQ4. The studied research questions do not influence each other. The fault rejection results are not affecting how much the tail latency is changed by saturation in the different coding languages.

Each question asks how one outcome depends on one variable, the language. The language has to be the only planned difference between the two objects under test. In that form two programs written on their own are measured, and the difference in the results stem from the language. By building both versions from one set of specification, choosing the same design strategies and similar libraries further improves the comparability of the obtained results. For RQ4 the prototype measures the application programming interface (API) between the two languages and the code behind it. Prior work says designing that interface has a cost. This study does not measure it. A longer study would have to measure it.

The closest prior work is the five vendor-hosted theses of Section 2.3. The wider literature came from the digital libraries of the Association for Computing Machinery and the Institute of Electrical and Electronics Engineers, the proceedings of the USENIX Association, and Google Scholar. The searches combined the terms *Rust*, *C++*, *memory safety*, *tail latency*, *systems programming*, *performance comparison*, and *RAN* or *5G control plane*, and citation chaining extended them. The five theses appear in none of the citation databases, neither Crossref, OpenAlex nor Semantic Scholar, so chaining cannot reach them. They were found by searching the university repositories directly, the Digitala Vetenskapliga Arkivet portal for the Nordic universities and Finna for the Finnish ones. Finna returns no thesis on Rust in a radio-access or telecom setting. An affiliation search of the vendor's indexed works in OpenAlex finds one that mentions Rust, an unrelated preprint. No search returned a study that measures the tail latency of a Rust and a C++ version of the same stateful, multi-thread control-plane part under too much load.

Table 3.2 summarizes the content of the rest of this chapter. Each row is one research question. The columns say which test answers it, what was built for that test, which data the test produces, and which number is reported.

RQ1 asks how long a call takes, and RQ3 asks how much processor time and memory the service uses. H1 is the hypothesis for both questions. It says the two languages are performance-equivalent below saturation, with any difference only among the slowest calls. One hypothesis then has to hold for the delay and for the resources together, which is why that wording is its main weakness. The two need not move together. Chapter 6 returns

to that wording.

RQ2 records whether the compiler accepts the program and what the C++ sanitisers report. These results are independent of the machine. Most of RQ4 consists of sizes or counts, which are likewise machine-independent. Build time is measured with a stopwatch. A faster machine reduces both the Rust and C++ build times, so absolute times depend on the machine used. To control this, the study reports the ratio of the two build times, measured sequentially on the same machine. Thus, the comparison is made within the same machine, rather than across machines.

Table 3.2: How each research question is answered.

RQ	Method	Artefact / data	Primary metric
RQ1	Load sweep to overload, interleaved repeats	Benchmark harness, latency histograms	median, p99/p99.9/p99.99, and maximum latency vs. offered load
RQ3	Same sweep, resource capture	`getrusage` samples per run	peak resident set size (RSS), CPU, throughput
RQ1, RQ3	Core called directly, then through the service	Matched microbenchmark pair	operation vs. cross-thread handoff
RQ2	Controlled fault injection	Defect catalogue, three C++ sanitisers, `rustc`, Loom/Shuttle, Miri	compile-time / runtime / undetected
RQ4	FFI migration prototype	C++ host + Rust C-ABI boundary	boundary size, `unsafe` scope, per-call cost

3.4 Methods per research question

RQ1 and RQ3 use a load sweep from a light load to well past saturation. Each producer sends on a fixed schedule and every delay is timed from that schedule [37]. The four builds take turns inside each repeat. The order is always the same, with C++ first. So a slow drift in the machine reaches all four, while a brief disturbance reaches only the build that is running.

The time from send to reply mixes two costs. The core operation is the work done on the device record. The handoff passes that work to the worker thread and brings the answer back, through a queue and a reply handle that each build implements in its own way

(Chapter 4). A second timing measures those two costs apart. The round trip spends most of its time on the handoff, so the split shows how much of the measured time the core operation can account for.

A third timing measures the bounded queue and the reply handle on their own, without any `ShardCore` work in the path. The two languages build these pieces differently, so this test focuses only on the submit-and-wait synchronisation between the producers and the worker. Several producers compete for the same bounded queue, and each waits on a one-shot handle until the worker signals back. Together with the core-versus-handoff split above, this helps narrow down which part of the path may contribute to the overload delay.

Past saturation, the sender records extra delay samples for the requests it fell behind on (Section 5.2). The reported campaign is the full set of timing runs this thesis reports. In that set the sender records those samples while the timer runs. Those overload comparisons are among the planned tests. A follow-up run on the same host writes them after the timed window, so that the recording cannot slow the sender. It is reported as a check, not as one of the planned tests.

RQ2 is answered by controlled fault injection [38]. Each bug class is a small program of its own, written in both languages. The C++ side is built and run under AddressSanitizer, ThreadSanitizer, and UndefinedBehaviorSanitizer. Those three tools watch a running C++ program for memory bugs, thread bugs, and undefined behaviour. Undefined behaviour is a mistake the language does not define, so a running program may do anything. The safe-Rust side is compiled. Where it compiles, it is run with a time limit, and the run is stopped if it takes too long. The lock-order deadlock and the lost update are also rewritten as small test programs for two Rust tools, Loom and Shuttle, that try many thread orders. The case inside `unsafe` is run under Miri, an interpreter that checks Rust for undefined behaviour. No static analyser was run on either side. A static analyser reads the source without running it, and some C++ analysers flag some of these patterns at build time (Figure 5.8 names one). The C++ columns therefore show the least that C++ tools can catch, and RQ2 compares the Rust compiler with these three sanitisers.

RQ4 is answered by a working prototype in the direction a real migration takes. A C++ host calls an unchanged Rust core across a C ABI. That thin layer converts between the C++ host and the Rust core. The per-call cost is measured on a real signalling operation. Memory bugs that cross a language sit in that layer [39].

3.5 Fairness as a controlled variable

A language comparison is only as good as what it keeps the same. The specification is fixed before either version was implemented. Several choices are then matched on both sides, and never tuned to favour one language. Those choices are the thread model, the allocator and its number of memory pools, the opt level, the hash function, the histogram method, and the load schedule. The opt level is how hard the compiler tries to make the code fast. The allocator splits its memory into separate pools, called arenas, so that threads rarely wait for each other, and each pool keeps some spare memory. Both sides use the same speed setting and the same number of pools, although the two `jemalloc` builds are different releases (Chapter 4). Where the two languages could not be written the same way, the gap is written down and left in place.

The two sides are shown to match by comparing their logs, operation by operation. For a given seed they also run the operations in the same order. A seed is the starting number of that random order, so the same seed repeats the same sequence. The four builds take turns on the same pinned cores inside each repeat. A pinned core is reserved for the test, and no other work is allowed to use it. Running the same work one build after another reduces differences caused by changes over time in the processor's recent memory and branch guesses. The cache is the small fast memory next to the processor. A branch guess is the processor's guess about which way a condition will go. The test neither empties the cache nor lets the processor practise those guesses before each build. The work and the cores stay fixed, but the processor's recent memory and branch history do not.

3.6 Statistical treatment

This section explains how the statistical tests were chosen and how the results should be interpreted.

All statistical tests were decided before the experiment was run. This is important because it prevents choosing tests after seeing the results. For example, we did not add an extra load point to the statistical analysis just because it looked interesting in the load sweep.

The study tested 14 planned comparisons between C++ and Rust. In each comparison, the allocator was kept the same. This means that the comparison measured the difference between the languages, not the difference between allocators.

At 56 kilo-operations per second (kops/s), the tested measurements were:

- median latency,
- p99 latency,
- p99.9 latency,
- peak RSS,
- CPU use.

These measurements were tested under both allocators. At 784 kops/s, the study tested sustained throughput and CPU use under both allocators. The full list of planned tests is shown in Tables A.3 and 5.2.

Each test compared C++ and Rust repeat by repeat. This is called a paired test. It is useful because both languages are compared under nearly the same conditions. For example, if the machine is temporarily slower during one repeat, both C++ and Rust are affected.

The study used a paired permutation test. In simple terms, this test asks:

> Could the observed difference between C++ and Rust have happened by chance?

To answer this, the test repeatedly swaps the C++ and Rust labels within each paired repeat and recalculates the difference. If a difference as large as the observed one is rare after these swaps, the result has a small p-value. A small p-value means that the observed difference is unlikely to be due to chance alone.

The study also reports a bootstrap confidence interval for each tested comparison [40]. The confidence interval shows the uncertainty around the measured difference. In simple terms, it gives a plausible range for the real difference between C++ and Rust.

Because many tests were performed, the study used the Holm correction [41]. This reduces the chance of wrongly reporting a difference just because many comparisons were tested.

The number of repeats also limits how small a p-value can be. The smallest possible uncorrected p-value is 0.0000019, and the smallest possible Holm-corrected p-value is 0.000027. These limits are reported so that readers can see when the test has reached its resolution limit. In this study, a *tie* means that the test did not find a statistically detectable difference between C++ and Rust.

Some measurements were not statistically tested. They are reported only to help describe the system's behaviour.

- **Throughput at the selected load** was not tested because the experiment already fixed the request rate.
- **Jitter** was not tested because it was only used to describe how much latency varied from one call to the next.
- **Maximum latency** was not tested because it depends on only one call. One unusual call can therefore decide the whole value.
- **Pooled p99.99 latency** was not tested because it is calculated by combining all calls into one distribution. The statistical test instead uses paired repeat-level measurements.
- **Latency percentiles after saturation** were not tested because the sender can no longer maintain the planned request rate. The measured latency then includes both service time and extra waiting time caused by the sender falling behind. For this reason, these values are not a fair measure of processing time after saturation.

3.7 Reproducibility and research artefacts

The code, raw logs, and scripts required to rebuild the reported results are available at https://github.com/VienThanh12/Master-Thesis. Access to the repository is restricted during review. Therefore, the same package is also submitted with this thesis. The results can be rebuilt from either copy without access to Nokia's private code, since the experiments are driven by the public fixed specification.

The package contains the benchmarking harness, the fault catalogue and runner, and the FFI prototype. Measured plots and quoted numerical results are regenerated directly from the raw logs using `bench/refresh_thesis.sh`. The raw logs are included in the package, and each campaign has its own script and macro prefix to keep campaign results separate.

Code size, complexity, and binary footprint are measured from the source tree rather than from the logs. These measurements describe the version used during the campaign, before the probe options and asynchronous interface added in October. The reproducibility claim does not cover hand-drawn schematics, which are explanatory rather than measured results.

For each run, the host setup is recorded and stored with the corresponding campaign data. Running the same scripts on data from another host follows the same procedure but may produce different numerical values. No results in the thesis are copied by hand. They are generated from the stored data by script.

4 Design and Implementation

Nothing is built until the specification is fixed. It fixes the public API, the UE state machine, the data layout and the steps on it, the locks and shard size, and the thread model. Both versions follow it. A gap between them then comes from the language and the idioms it forces, which Table 4.1 logs, rather than from two different designs. The specification is itself a result of the study. It is the main guard against favouring one language while writing either version.

One thread model is used on both sides: shared-nothing and thread-per-shard. Shared-nothing means one thread never reads another thread's records. Thread-per-shard means each slice of those records has one owner thread. That layout is known to change tail latency by itself [42]. The layout is held fixed as a fairness control. A production gNB-CU-CP schedules UE work differently, so this layout is a choice of the study and not a copy of production. In it, the id space is split into shards. Each shard is owned by one worker thread. That thread alone reads or changes it. So no lock guards context state. Callers send a request to the owning shard's bounded queue. They wait on a one-shot reply handle. The queue blocks when it is full. That is on purpose. A drop rule would hide overload. This study needs overload to show up as latency. The only shared mutable state either version allows is one atomic counter. No control flow depends on its value. Figure 4.1 shows the data flow. There is one multi-producer, single-consumer (MPSC) queue per shard. Its worker drains it first in, first out (FIFO).

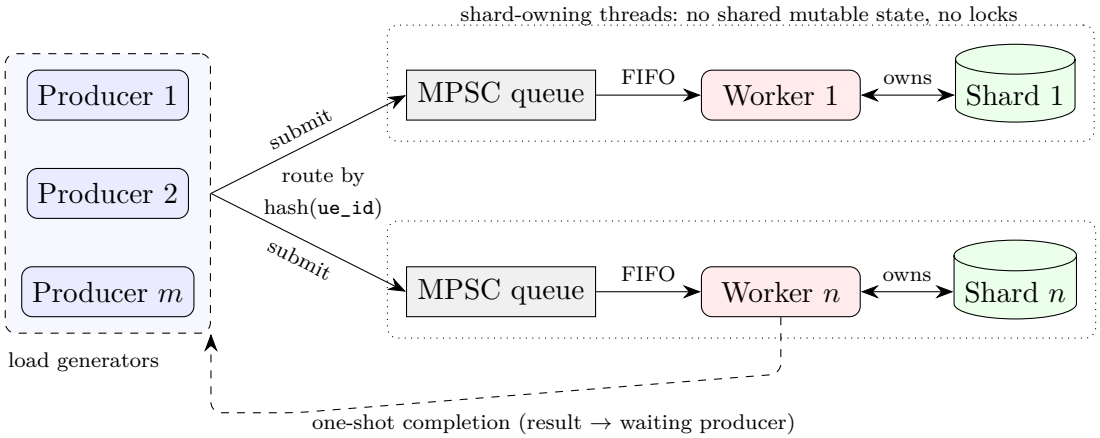


Figure 4.1: Shared-nothing, thread-per-shard architecture. Producers submit to a bounded MPSC queue per shard, and that shard's worker drains it.

Rust

```rust
pub enum Request {
    // Create is left out here.
    Setup {
        ue_id: UeId,
        stamp_completion: bool,
        resp: CompletionTx<Completion<OpResult<()>>>,
    },
    Signal {
        ue_id: UeId,
        msg: SignalingMsg,
        stamp_completion: bool,
        resp: CompletionTx<Completion<OpResult<SignalingAck>>>,
    },
    // AddDrb, BeginReconf, CompleteReconf, AbortReconf,
    // Release, Lookup and Count are left out here.
    Stop,
}
```

C++

```cpp
using Response = std::variant<
    OpResult<void>,
    OpResult<SignalingAck>,
    OpResult<UeContextView>,
    OpResult<std::uint64_t>>;

struct Request {
  OpKind kind{};
  UeId ue_id = 0;
  CreateParams create_params{};
  SignalingMsg signaling{};
  DrbEntry drb{};
  ReconfigParams reconfig{};
  bool stamp_completion = false;
  std::promise<TimedResponse> completion;
};
```

Figure 4.2: The request types, taken from rust/src/message.rs and cpp/include/uecm/message.hpp, are shown with comments, probe-only code, and the TimedResponse struct removed. In Rust, each request variant, except Stop, contains only the fields needed for its operation and a reply channel typed to that operation's result. In C++, each request stores fields for all operations, making it larger: 160 bytes versus Rust's 96. C++ sends replies through a std::promise, where TimedResponse contains a Response variant covering all possible results.

Fairness is kept below the design as well as in it. Both builds link the same `jemalloc` and keep a system-allocator build as a control. Both compile at the same optimisation level, `-O2` for `g++` and `opt-level = 2` for Cargo, and both record latency through one high-dynamic-range (HDR) histogram set up the same way [43]. Hashing is pinned on both sides. SplitMix64 assigns the shard [44], and FNV-1a, the Fowler–Noll–Vo hash [45], is used inside the map. C++'s default hash of an integer is often just the number itself, whereas Rust's is strong on purpose. Left as they came, the two standard libraries would have been the measurement.

Table 4.1: Where C++ and Rust could not use the same design. Each row is one requirement and how each language meets it.

Concern	C++	Rust
Result type	`std::expected<T, OpError>`	`Result<T, OpError>`
Request type	Tagged struct with every operation's fields, 160 bytes	`enum`, each variant with only its own fields, 96 bytes
Reply type	A `std::variant` of four results	Each operation's own result type
Bounded queue	Hand-written in 51 lines from a mutex and two condition variables	Standard `sync_channel(capacity)`
Reply handle	`std::promise` and `std::future`	One `mpsc::channel()` per request
Per-shard map	`std::unordered_map` with `FnvHash`	`HashMap` with `FnvBuildHasher`
Shared counter	`std::atomic<uint64_t>` by reference	`Arc<AtomicU64>`, cloned per worker
Not running	A call throws `std::logic_error`	A call panics
Teardown	The destructor calls `shutdown()`	`Drop` calls `shutdown()`
Optimisation	`-O2`	`opt-level = 2`

Where the languages cannot meet a need the same way, the gap is written into a forced-idiom log, which Table 4.1 lists. Two of those gaps were candidates for the overload lead of Section 5.3.2. The first is the bounded queue. Rust uses `sync_channel`. C++23 still has no matching primitive, so the C++ build implements the same blocking semantics with a mutex and two condition variables. The second is the one-shot reply handle. C++ pairs `std::promise` with `std::future`. Rust allocates a fresh `mpsc` channel for each request. Section 5.3.2 replaces them on further hosts, first the queue alone and then the queue and the reply together. A third carries the core-operation result of Section 5.3.1: the per-shard container is each language's own standard hash map. A fourth is the request encoding. C++ has no payload-carrying enumeration in this idiom, so a request becomes a tagged struct holding each field any operation might need, 160 bytes against Rust's 96. The net result is a Rust core 20% longer in code lines. Its branching complexity is 13% lower [46]. The Rust library contains no `unsafe` at all, as Section 2.2 defines safe Rust.

Parity is tested and not assumed. Language-neutral vectors run in both versions. Their logs are diffed operation by operation. A cross-diff cannot tell "both correct" from "both wrong in the same way". A third version in Python acts as an independent oracle. An oracle is a third program, written on its own, whose answers the other two have to match. It has to reproduce all 223 hand-derived expectations before it may generate any of its own. A set of 9 000 generated operations found no mismatch. The C++ side runs under AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer. The Rust side runs under Miri, an interpreter that checks `unsafe` Rust for undefined behaviour. A clean sanitiser result is bounded by the code that actually ran.

5 Experimental Setup and Evaluation

This chapter first describes the machines used in the experiment and the workloads that were run on them. It then presents the experimental results. First the experiments about the impact of increasing load on the component are described, followed by experiments for the remaining research questions. Several terms carry a specific meaning throughout this chapter.

Offered load is the rate at which the test sends requests to the service. For example, an offered load of 10,000 requests per second means that the test attempts to send 10,000 requests every second.

Saturation is reached when the service can no longer keep up with the offered load. Past this point, a higher requested rate does not raise the rate at which the service finishes work, because work begins to accumulate or the sender begins to fall behind.

Median latency is the latency at the middle of the measured observations. Half of the calls are faster than the median and half are slower.

Tail latency is the slow end of the latency distribution. It describes unusually slow calls rather than the typical call. Percentiles such as p99 and p99.99 name this part of the distribution.

5.1 Testbed setup

Some numbers in this chapter come from the tools and the code. Other numbers depend on the machine they ran on. The reported campaign comes from one bare-metal host, a single-socket AMD EPYC 7443P. Bare metal means the program runs on the machine itself, with no virtual machine in between. Each core runs one thread. Extra threads on a core are off. The host runs Ubuntu 24.04.4 LTS on kernel 6.17.0-23-generic. Linux can raise and lower each core's clock as load and heat change. For the experiments, the core's clock is held constant, by disabling performance and boost settings. The test may use only cores 2–17. There is no hypervisor. A hypervisor is the program that would run a virtual machine. On those cores the kernel turns off its timer tick (`nohz_full`). If left on, those ticks would show up in the latency numbers. The study does not use another

Linux setting, `isolcpus`. That setting would stop Linux from moving threads between the allowed cores. Every test thread would then stay on the first of those cores and wait its turn there. Without the setting, Linux spreads the test's threads across the allowed cores.

The compiler and build tools used in the experiment are not taken directly from the host system. Instead, both the C++ and Rust programs are built and executed inside the same container image. The image is based on Rocky Linux 8.10 (Green Obsidian) and provides `g++ 15.2.1` and `rustc 1.97.1`. This ensures that the build environment is controlled and reproducible rather than depending on the software installed on the host machine. The image specifies the major version of the C++ compiler, but not its exact point release. As a result, the compiler version used in this experiment is `g++ 15.2.1`. A container does not completely isolate the experiment from the host system. It still uses the host's Linux kernel.

Until late in the study, the load sweep was performed inside a Kernel-based Virtual Machine (KVM) running on an Intel(R) Xeon(R) CPU E5-2680 v4. The CPU core controls described above were not applied during this earlier sweep. The experimental notes record the hypervisor as kvm, the guest operating system as Rocky Linux 8.10 (Green Obsidian), and the guest kernel as 7.0.2-1.el8.elrepo.x86_64. Here, the guest refers to the virtual machine itself, while userland refers to the programs and libraries that run on top of the operating-system kernel. The earlier sweep showed substantial variation in the measurements. For one variant, the 95% bootstrap confidence interval for p99 latency had a width of 1480 μs. Across the four variants, the difference between the smallest and largest p99 values was only 293 μs. The uncertainty in one variant's p99 measurement was therefore larger than the spread among all four variants, so those ranked differences cannot be read with confidence. This earlier sweep is nevertheless retained because it used the same experimental design as the main experiment. That retention makes a side-by-side comparison possible. The purpose of this comparison is to examine whether the observed findings are consistent across different environments and therefore less likely to be specific to one particular machine or environment.

A third sweep was performed on a second bare-metal machine with an AMD EPYC 7443P processor. This sweep changed several experimental choices at the same time: the shard count, the producer count, the set of offered loads, the number of repetitions, and the tools used, and it ran without `nohz_full`. It therefore provides a broader robustness check. Specifically, it tests whether the findings remain similar when several experimental choices are changed together. It does not isolate the effect of any one of these changes separately.

Appendix A provides a field-by-field description of the three hosts and their corresponding experimental settings.

5.2 Workload and measurements

The test sends a list of requests, which were designed during this study. The requests follow the 3GPP rules for one device's connection [1, 4]. The driver is the program that sends the list. It takes each device, called a UE, through the steps.

The *signal* list is one workload. The driver connects a group of devices, then sends the same signalling request many times. That repeated request is the common path. The *mixed* list is the other workload. Each device goes through the whole connection. The service makes the device's record, starts the connection, handles a signal, adds a data channel, changes the connection or moves the device to another cell, and then ends the connection. In the test those steps are called context creation, setup, signal, bearer setup, reconfiguration or handover, and release.

In the mixed list the order of the steps is random. A starting number, the seed, fixes the order, and the same seed repeats it. With one seed, the C++ version and the Rust version get the same list. While the service keeps up, both finish the list. Each kind of request, such as a setup or a release, then happens the same number of times in both. The two versions did the same work. When the service falls behind, the run reaches its time limit before the list is finished. The two versions can stop in different places, so the counts can differ.

The main tests and the follow-up use the mixed list. The signal list is used only when several requests can be waiting at once (Section 5.3.2). Figure 5.1 shows both lists, and the steps of the mixed one.

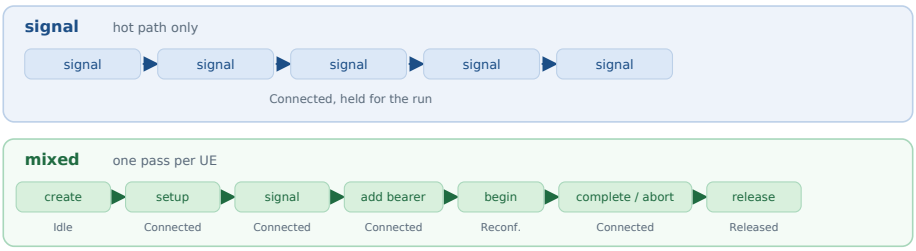


Figure 5.1: The two work loads used in the experiments of this thesis. The *signal* workload only repeats signaling of connected UEs. The *mixed* workload goes through the whole lifecycle of a UE.

The 3GPP specifications do not define how many times each procedure should be triggered

by a cell. In a live network, the frequency of cell changes depends on factors such as site location and time of day, and there is no published standard rate for this behavior.

As a result, the traffic mix used in this test may differ from what occurs in a real cell. However, all four builds are tested with the same mix. While the mix may affect the absolute values of the results, it affects all builds equally, so the comparison between builds remains fair.

Each producer sends its requests on a planned schedule. It sleeps until a request's planned time, sends it, and waits for the reply. If it is already late, it sends the next request at once, so it has at most one request at a time. Every delay is counted from the planned send time.

Below saturation the producer is asleep until that planned time. It runs at ordinary priority, so Linux may wake it up to 50 μs late [47]. That late wake-up is part of every delay this clock records (Section 5.3.1). Past saturation the producer falls behind the plan. The test then writes extra delay samples for the requests it should have sent, and in the reported experiment it writes them while the timer is still running. The timer then measures that writing as well as the service (Section 5.3.2).

Those extra samples are this test's histogram fill. HdrHistogram adds them so a sender that falls behind still records the requests it missed [37, 43]. The missed requests are the error called coordinated omission. The fill adds up to 10 000 extra samples for each finished call. Histogram fill is this harness's name for that step. It is not a standard benchmarking term.

Each load level is tested 20 times. The four language and allocator variants are interleaved within each repeat, in the same order every time, to reduce the effect of changes in the test environment. Each run lasts 3 s after a 1 s warm-up, and past saturation a run stops at 6 s. Each reported value is the median of the measurements, together with a bootstrap 95% confidence band [40, 48].

The planned comparisons below saturation, and the resource plots, use one fixed load, 56 kops/s. That number comes from the virtual machine, 56 kops/s, and it was fixed before this experiment ran. An earlier rule chose the highest load at which every build's slow-call time, its p99, stayed under 5 ms. On the virtual machine the noise reached that 5 ms line, so the rule stopped there. On this quieter computer the same rule climbs until the service can no longer keep up. That is a different load, the saturation point. The old rule is kept only as a check.

Every build on this computer keeps up with the offered load through 392 kops/s. The fixed load is below that point. At 448 kops/s, Rust with `jemalloc` still keeps up. The

other three builds do not.

5.3 Results

The safe-code results and most migration results do not depend on the host used. However, the speed and RSS results do. The numbers below come from one computer, unless the text or a caption names another. That computer ran the reported campaign. It is the bare-metal AMD EPYC 7443P in Section 5.1. The second bare-metal host and the further hosts A to D are named wherever their results appear.

5.3.1 Below saturation (RQ1, RQ3)

This part compares the two languages while the service is still able to keep up with the offered workload. At the chosen load of 56 kops/s, all four builds remain sustainable. Each can process the incoming requests without falling behind.

At this load, the four median latencies differ by at most 1.6 μs, which is 2.2%. A paired test still finds a gap of up to 1.5 μs between the languages. Rust has slightly lower median latency than C++ under both memory allocators. Figure 5.2 shows the load sweep behind that number.

p99 is a slow call. In 100 calls, 99 were faster. The test is repeated, so each build's p99 has a margin of error, and on this computer that margin is at most 0.8 μs. The fastest of the four builds and the slowest are 1.8 μs apart. The margin is too small to close that gap.

One comparison may show a difference purely by chance. When 14 planned comparisons are made, the chance of observing such random differences increases. The Holm correction accounts for this by retaining only those differences that are unlikely to have occurred by chance anywhere in the full set of planned comparisons. After applying the correction, both p99 comparisons and both p99.9 comparisons remain significant. In each of the 4 significant tail-latency comparisons, Rust has lower latency. This is consistent with the median results: the performance difference between languages is visible both for a typical call and for the upper tail latency. Table A.3 lists all planned comparisons at this load.

Table 5.1 shows the p99.9 and the maximum at the chosen load. The maximum is the slowest call in a run, and the first research question, RQ1, asks about it. C++ and Rust are further apart there than at any percentile in the table, and Rust is the lower of the two. The C++ and Rust intervals do not overlap. The planned comparisons leave the

maximum out, because one call slowed by a disturbance can set it for the whole run. The pooled p99.99 is left out as well.

Table 5.1: Delay of one call at the chosen load, in microseconds, timed from the planned send. The number above a bracket is the median, and the bracket is the 95% interval. † The p99.99 column is pooled, so it has no interval.

			(µs)		
Variant	p50	p99	p99.9	p99.99†	max
C++ (system alloc)	73.9	76.9	81.0	110.0	226.4
	[73.9, 74.0]	[76.4, 77.2]	[80.7, 82.0]		[221.1, 239.3]
C++ (jemalloc)	73.8	77.1	80.2	143.0	227.8
	[73.7, 73.9]	[77.0, 77.1]	[79.9, 80.9]		[217.2, 244.4]
Rust (system alloc)	72.7	75.3	77.1	87.9	100.3
	[72.6, 72.7]	[74.9, 75.6]	[76.9, 77.9]		[97.4, 160.8]
Rust (jemalloc)	72.3	75.7	77.8	81.3	104.4
	[72.2, 72.3]	[75.6, 75.8]	[77.6, 77.9]		[94.1, 141.8]

The slowest calls show the same separation between the languages. The runs' histograms are pooled, which gives 3 360 000 to 3 360 063 calls per variant. The two Rust variants sit below both C++ variants. The widest pair differs by 62 μs. The slowest Rust variant still sits 22 μs below the fastest C++ one. A second computer also runs the test without a virtual machine, but it is set up differently (Section 5.1). It shows the same pattern for the slowest requests. Here that means the delay that 9,999 of every 10,000 requests stay under. At loads from 32 to 96 thousand requests per second, that delay is 147–228 μs in the C++ builds and 88–115 μs in the Rust builds. The two ranges do not overlap. Rust is faster there. At lower loads the two languages cannot be told apart. The highest load on that computer, 128 thousand requests per second, is left out. At that load the typical request got faster, while the slower requests did not change, and the data from that computer do not explain why.

Two further limits belong to the test. A different computer would not remove them.

The first is what the timer includes. The delay is counted from the planned send time, while the producer is still asleep, and Linux may wake it up to 50 μs late [47]. That late wake-up is part of every delay in Table 5.1. The virtual machine was timed in two ways. In the main test the typical delay was about 80 μs. In a separate test the next request went out as soon as the reply came back, so there was no wait for a planned time, and a round trip took 13.62 μs. The late wake-up is probably also why the delay in panel (a) of Figure 5.2 drops a little as the load rises. At a higher rate a producer sleeps less often. Each language sleeps with its own function. A small gap between C++ and Rust at this

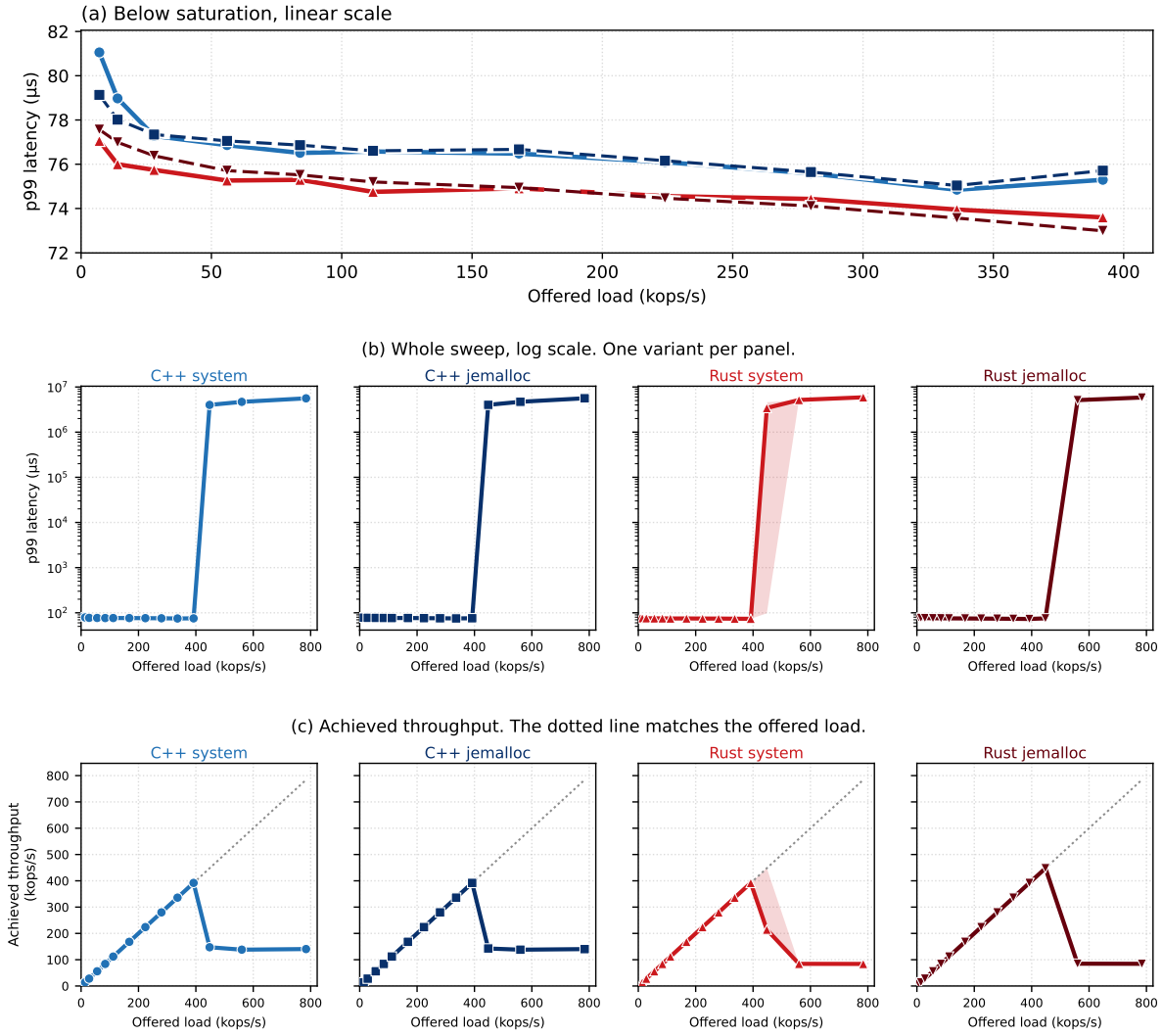


Figure 5.2: Panel (a) is p99 below saturation, on one linear scale, through 392 kops/s. The four variants sit within a few microseconds, so they are drawn together: blue lines are C++ and red lines Rust, solid for the system allocator and dashed for `jemalloc`. In (b) and (c) each variant has its own panel. Panel (b) is tail latency on a log scale across the whole sweep. Panel (c) is achieved throughput, and the dotted line matches the offered load. Lines are medians over 20 repeats, and the bands in (b) and (c) are bootstrap 95% confidence intervals. Past the knee, (b) and (c) show the reported campaign, whose sender records extra delay samples inside the timed window, so they time that recording as well as the service (Section 5.3.2).

load may come partly from those two functions.

The second limit is how little of a request is real work. That work, the core operation, reads or changes one device's record. A separate timing found that it takes at most 0.250% of a round trip. The rest is the handoff. The request waits in a queue for the worker, the answer comes back, and Linux puts threads to sleep and wakes them. When the real work is this small, passing the request between threads takes most of the time [49]. Figure 5.3 shows the split.

The gap between the languages in the typical delay is up to 1.5 μs. That is many times the whole core operation. A core operation that took no time at all would explain only a small part of the gap. Most of the gap comes from the handoff, which the two versions build differently (Section 5.3.2), or from the producers' sleep. This test cannot say which language does the core operation faster (Figure 5.4). Timed on its own on the virtual machine, the core operation is 1.6 times faster in Rust. The study did not find out why. One possible reason is the hash map each language uses to store the records. Another is the machine code each compiler produces.

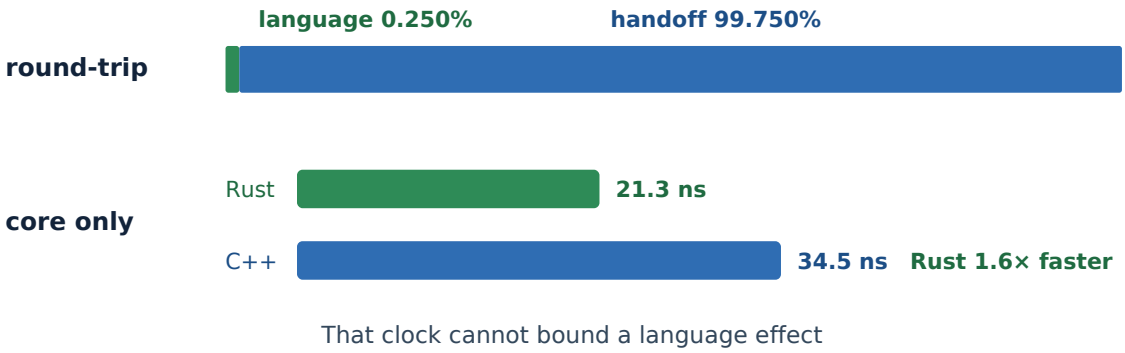


Figure 5.3: Where the time of one round trip goes, from the separate timing of its parts on the virtual machine of Appendix A. The core operation, labelled language in the drawing, takes 0.250%. The lower bars are that call timed alone, 21.3 nanoseconds in Rust and 34.5 nanoseconds in C++. A clock that spends the rest on the handoff cannot bound a difference in the core operation. The campaign's own clock also holds the producer's late wake-up, which this split leaves out.

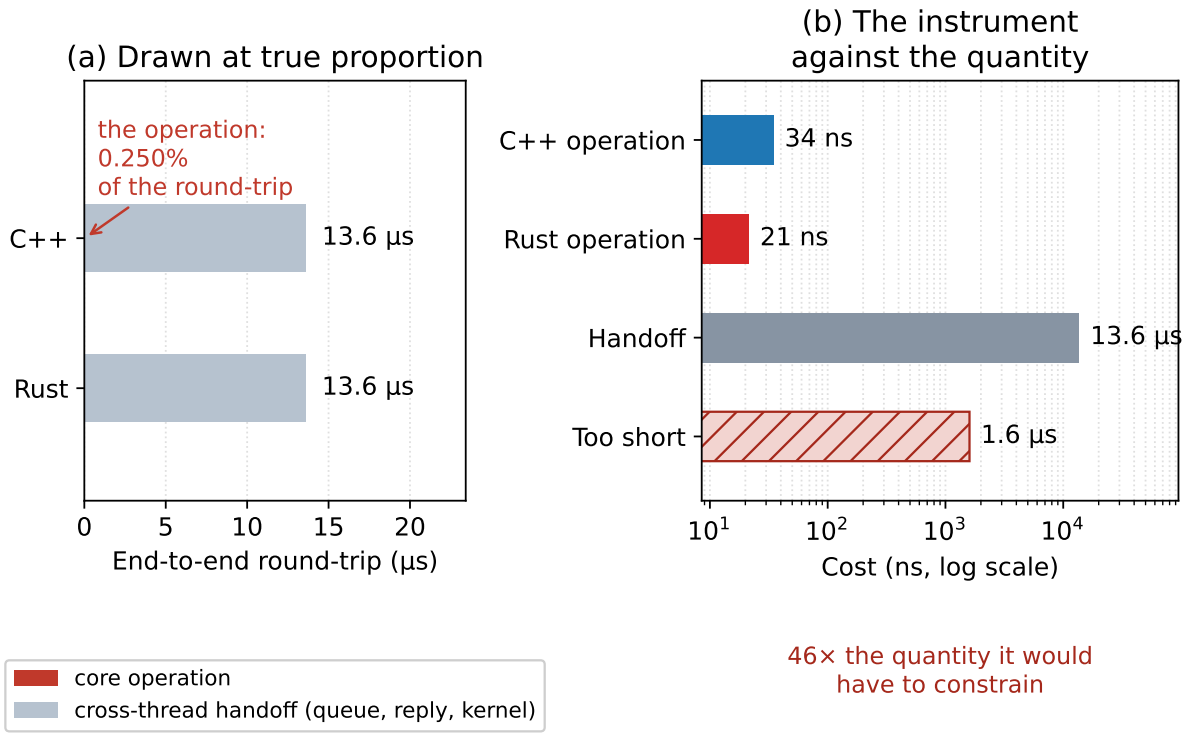


Figure 5.4: Where an end-to-end round trip's cost goes. **(a)** The round trip at true proportion, from the same separate timing on the virtual machine of Appendix A. **(b)** The operation costs from that timing beside the widest bound of this campaign's median comparison, 1.6 μs, which is 46 times the whole operation, so that comparison cannot resolve a difference in it.

5.3.2 Past saturation (RQ1)

Past saturation the answer depends on what the timer includes. The planned comparison on the reported campaign has C++ finishing more calls, but that clock also times the sender's own recording. Once the recording moves outside the timed window, Rust finishes more calls on both allocators. That lead comes from the handoff each build uses to pass a request to the worker and the reply back, not from the core operation. With several requests in flight, which language waits less depends on the allocator.

The planned comparison. At 784 kops/s the planned comparison has C++ finishing more calls than Rust. The ratio is 1.7, with C++ at 140 kops/s and Rust at 84. It holds in 20/20 paired repeats under each allocator, and it passes the Holm correction (Table 5.2). Panel (c) of Figure 5.2 draws it. This timer does not time the service alone. When the service falls behind, the sender writes extra delay samples for the requests it missed (Section 5.2). In the reported experiment it writes them while the timer runs, so part of the timed window is that writing.

Table 5.2: The four planned comparisons past saturation, Rust minus C++. The number above a bracket is the median difference, and the bracket is the 95% interval.

Metric	Allocator	Rust minus C++	p	Holm p
Sustained throughput (kops/s)	system	-56.6	0.0000019	0.000027
		[-57.2, -56.1]		
Sustained throughput (kops/s)	jemalloc	-55.9	0.0000019	0.000027
		[-56.3, -55.5]		
CPU utilisation (%)	system	+18.1	0.0000019	0.000027
		[+17.2, +19.6]		
CPU utilisation (%)	jemalloc	+17.0	0.0000019	0.000027
		[+16.6, +18.6]		

With the recording after the window. A follow-up on the same computer writes those samples after the timer stops. It uses the same reserved cores and the same `nohz_full` setting. It is a check, and it was not one of the planned tests. The order reverses. Rust keeps up to 448 kops/s, and C++ only to 392 (Figure 5.5). At 448 kops/s, C++ serves 416 kops/s and Rust serves 448. At the highest load, C++ finishes 0.95 of Rust's throughput on the system allocator ($p < 0.001$) and 0.92 under `jemalloc` ($p < 0.001$). CPU per finished call differs by a factor of 1.03, and C++ is higher. The large ratio in the planned comparison measures the recording more than the service.

The follow-up still understates Rust. The C++ harness stopped its timer before writing the extra samples, and the Rust harness stopped after, so Rust's window included more than its work. A timer test on host B measured the difference. Without the samples, Rust's throughput at the highest load rose by 2.4%, and C++'s did not move (Appendix B). Comparisons between the languages on this run therefore understate Rust's lead. Comparisons within one language are unaffected. Rust stays ahead, and the true lead is larger.

Where the lead comes from. On the follow-up, Rust finishes more calls because the whole trip is shorter. The sender waits for the reply before it sends again, so finished calls follow the whole round trip. The core work is at most 0.250% of that trip. At the highest load, 57 to 63% of a call's CPU is the kernel putting threads to sleep and waking them. C++ spends more of that time per call than Rust, on 14 of the 14 load levels and on both allocators, and 1.12 times as much at the top load.

Later runs change one piece of the build and repeat the comparison. They ran on further computers with the same processor (Table 5.3). Changing the C++ map leaves the lead in place. Giving Rust the C++ queue removes it, within the timer error above. That queue is one lock and two condition variables. Changing only the buffer behind the lock does

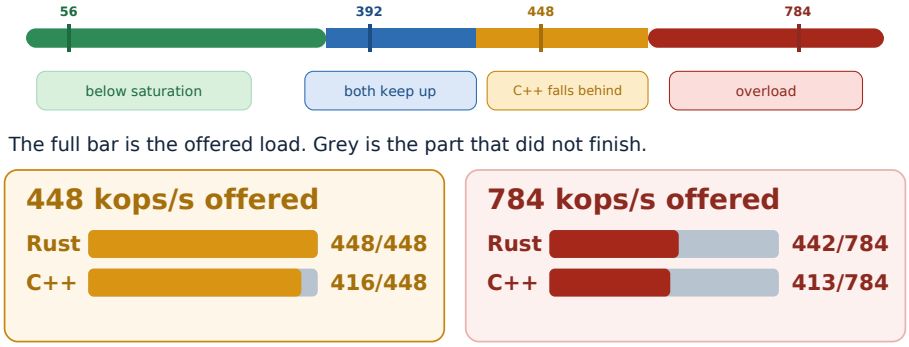


Figure 5.5: The figure shows four load points, ranging from light load to overload. These points are placed along the offered-load axis for easy reading. Their positions are not to scale. The tick marks show 56, the shared tracking load 392, the load where C++ first falls behind on this run at 448, and the overload point at 784 kops/s. The bars show finished throughput on the follow-up run, which writes the extra delay samples after the timed window and is not among the planned tests. Each bar's full length is the offered load at that point. The filled part is finished calls, and the grey part is offered load that did not finish.

not move C++. The lead comes from how the queue is locked and how a waiting thread is woken. In the C++ build that queue is this study's own code (Table 4.1), so the lead belongs to the two designs as written. It does not belong to either language in general. On the C++ path the kernel records 1.63 wake-ups per request that find no sleeping thread, against 0.19 for Rust.

One last run gives both languages the same hand-written queue and the same hand-written reply. Both wrap that queue around the same `ShardCore`, the code that reads or changes one device's record (Figure 5.6). With the system allocator, Rust's lead falls to 0.6% to 1.3% on hosts B and C. With `jemalloc` on host B, the test cannot separate the remaining gap from no difference (Appendix B). What can still differ is the code each language writes around the shared queue.

An earlier session timed the queue and the reply on their own, with no service around them, and could not tell C++ from Rust (Appendix A). The notes do not name the computer, or how it was set. Those timings cannot support these follow-up runs, and they cannot contradict them.

Where a call's time goes. Timing each part of a call on host C shows where the round trip goes (Table 5.4 and the bars in Figure 5.6). The queue and the reply take at least 97% of every call, split roughly in half. The worker's operation is at most 1.5% of a call. With each build's own queue and reply, Rust's call is 1.33 µs shorter, and 1.13 µs of that difference is in the queue. With the same handoff, the queue and the reply take the

Table 5.3: Follow-up runs past saturation. None of these runs is one of the planned comparisons.

Host	What changed	What happened to the Rust lead
A	C++ uses an open-addressed map, as Rust does	Stayed. Rust ahead in 10 of 10 repeats
A	Rust uses the C++ queue design, one lock and two condition variables	Gone. The swap closes 105% of the lead, and Rust is slower than with its own channel in 9 of 10 repeats
A	C++ keeps that lock with a fixed ring instead of a growable buffer	No change. C++ moved by 0.4%
B, C	Both languages use one hand-written queue and reply	Fell from 9.0% to 1.3% on host B, and from 7.8% to 0.6% on host C, with the system allocator

same time in both languages, to within 0.10 µs. Splitting the call adds two clock reads, so whole calls are compared on the runs without that split.

Delay with several requests in flight. Past saturation the sender falls behind. The delay above then includes two things: the service time and the time the sender is behind schedule. That delay does not by itself show which service is faster.

A separate experiment on host D therefore times the work itself. Several requests can be in flight at the same time. Each request keeps its planned arrival time, and timing stops when the worker finishes it. The offered load is 3920 kops/s. It is the first measured load where every variant completes fewer than 99% of its planned requests within their arrival windows. That rule was fixed before looking at which variant did better. The experiment is not one of the planned comparisons of the main sweep.

Table 5.5 gives the median for the signal workload and for the mixed lifecycle. With `jemalloc`, Rust is faster in every repetition for both workloads. With the system allocator, C++ is faster throughout. The difference comes from waiting to enter the queue, not from doing the work, and the time after entering the queue is shorter for Rust under both allocators. The two `jemalloc` builds are different releases (Chapter 4), so part of this split may belong to the allocator builds rather than to the languages.

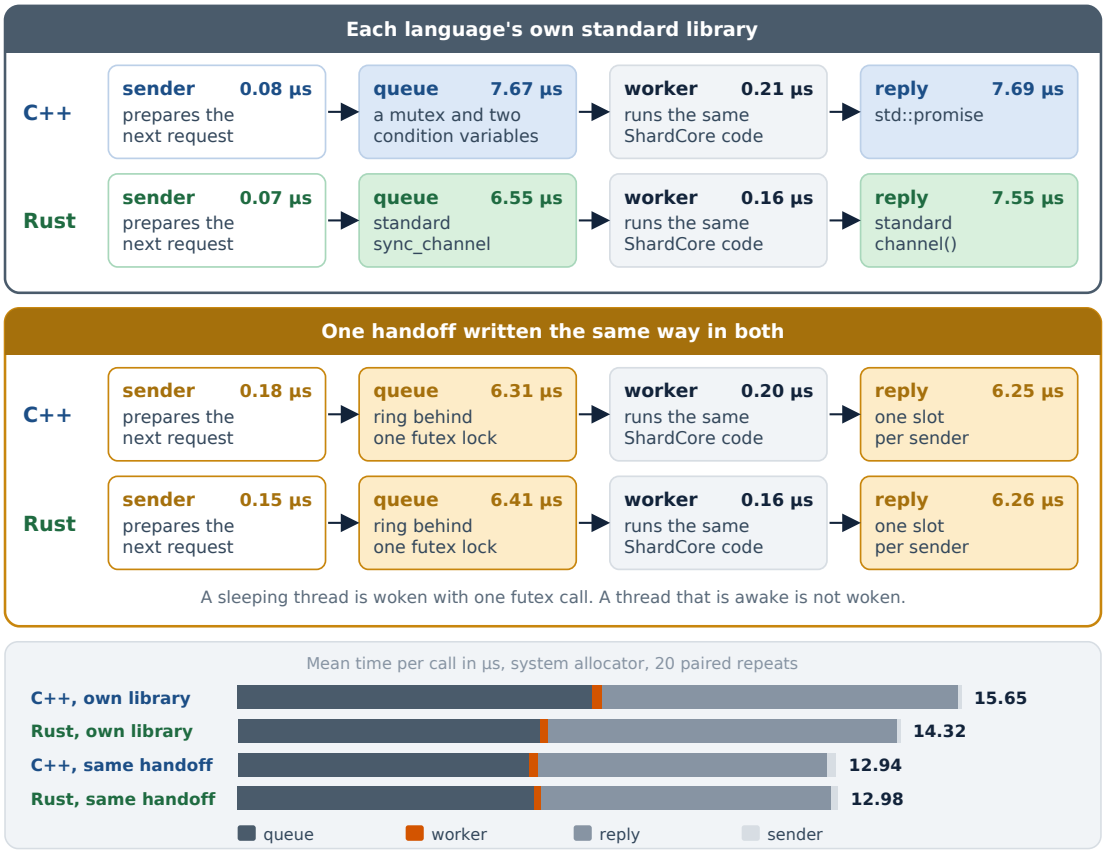


Figure 5.6: How a request reaches a worker, how the reply returns, and how long each part takes. The upper panel shows each build's own queue and reply, as in the reported service, where the C++ queue is the study's own code. The lower panel shows one hand-written queue and reply slot that both languages use. The worker runs the same `ShardCore` code in every row. Each box gives the mean time of its part, and the bars stack the parts into the whole call. The times come from host C, a further host with the same processor. This probe is not one of the planned comparisons.

Table 5.4: Time spent in each part of one call on host C, in microseconds, with the system allocator. The queue ends when the worker starts, and the reply ends when the sender holds the result. Rounded parts may miss the whole call in the last digit.

	Own queue and reply		Same handoff	
Part	C++	Rust	C++	Rust
Queue	7.67	6.55	6.31	6.41
Worker	0.21	0.16	0.20	0.16
Reply	7.69	7.55	6.25	6.26
Sender	0.08	0.07	0.18	0.15
Whole call	15.65	14.32	12.94	12.98

Table 5.5: Delay with several requests in flight on host D.

	Signal		Mixed	
	system	`jemalloc`	system	`jemalloc`
C++ median (ms)	200.7	121.9	199.8	120.4
Rust median (ms)	265.5	53.8	279.1	73.9
Rust faster	0/20	20/20	0/20	20/20
C++ in service (µs)	14.5	21.2	13.2	18.6
Rust in service (µs)	4.0	3.3	4.1	3.4

5.3.3 Resource use (RQ3)

The four variants have similar median latencies, but they do not use the same amount of RAM or CPU. The resource results show different patterns for memory and CPU, so the two are discussed separately.

Memory usage. Peak resident set size (RSS) shows two distinct effects: a *language effect* and an *allocator effect*. The language effect measures the additional peak RAM used by Rust compared with C++, while the allocator effect measures the additional peak RAM used by `jemalloc` compared with the system allocator. Table 5.6 reports both effects on all three hosts, and Figure 5.7 shows the results for the reported host at 56 kops/s.

The language effect is consistent across all three hosts: Rust uses more peak RAM than C++, and the difference remains within a few MiB. However, its exact size varies by host. In contrast, the allocator effect varies much more between hosts. On the virtual machine, for example, the allocator effect is about 13 times larger than the language effect. Thus, the relative size of the two effects is a property of the host environment, rather than a general property of Rust or `jemalloc`.

Table 5.6: Extra peak memory on each host, in MiB, as the smallest and largest difference. On the reported host each difference is a median of paired runs at the chosen load. On the other two hosts it is the difference between two medians, taken at each steady load.

	Reported host	Virtual machine	Second bare-metal host
Language effect	1.6–2.5	1.8–3.5	0.7–1.8
Allocator effect	1.5–2.3	32.5–34.3	1.9–3.2

The second bare-metal host provides more detail about the source of the additional Rust RAM. Table A.5 separates the process memory into allocator metadata, live application allocation, and allocator-resident memory. Each difference below is Rust minus C++. Arena

metadata differs by +2.07 MiB, live application allocation by -0.16 MiB, and allocator-resident memory by -1.92 MiB. Although the last two are lower, the peak RSS of the Rust process differs by +1.74 MiB. The metadata is counted inside the allocator-resident figure, so the extra metadata does not explain the higher peak. The two `jemalloc` builds also differ (Chapter 4), which may account for part of the metadata gap.

On that host, therefore, the higher Rust RSS is not caused by Rust holding more live application data. Instead, it is mainly a fixed cost of the process.

CPU usage. CPU usage does not follow the memory pattern. At 56 kops/s, Rust uses up to 6.2 percentage points less CPU than C++, and both CPU comparisons clear the Holm correction (Table A.3). Past saturation the planned CPU comparison has Rust drawing more (Table 5.2), but that clock includes the recording. On the follow-up run, CPU per finished call at 784 kops/s differs by a factor of 1.03, with C++ the higher one. A throughput lead remains there, and Section 5.3.2 traces it to how the queue is locked.

Hardware counter reading. The hardware-counter measurements on the second bare-metal host cover only an isolated call. There is no queue, no offered load, and therefore no competition between requests for shared resources. The measurements show how much CPU work is needed to process one request when it runs by itself.

For this isolated call, Rust retires 11% fewer instructions and completes in 15% fewer CPU cycles than C++. It also has 29% more cache misses per thousand instructions. Thus, Rust completes the isolated call faster even though its cache-miss rate is higher.

These measurements agree in direction with the isolated timing of Section 5.3.1, though not in size: 15% fewer cycles here against 1.6× there, on a different host with a different compiler pair. They describe how much work the faster build does, not which code saves it. However, the overload experiment is different: many requests arrive at the same time and must share the queue and its lock. Some requests therefore spend time waiting for the queue or for other threads to release the lock. This waiting does not occur in the isolated-core experiment and is not captured by the hardware-counter measurements. The overload experiment in Section 5.3.2 showed that the additional overhead comes from waiting for the queue lock and from the kernel time required for each handoff between threads. Therefore, the hardware-counter results explain the performance of an isolated call, but they do not explain the additional delays that appear when the system is overloaded.

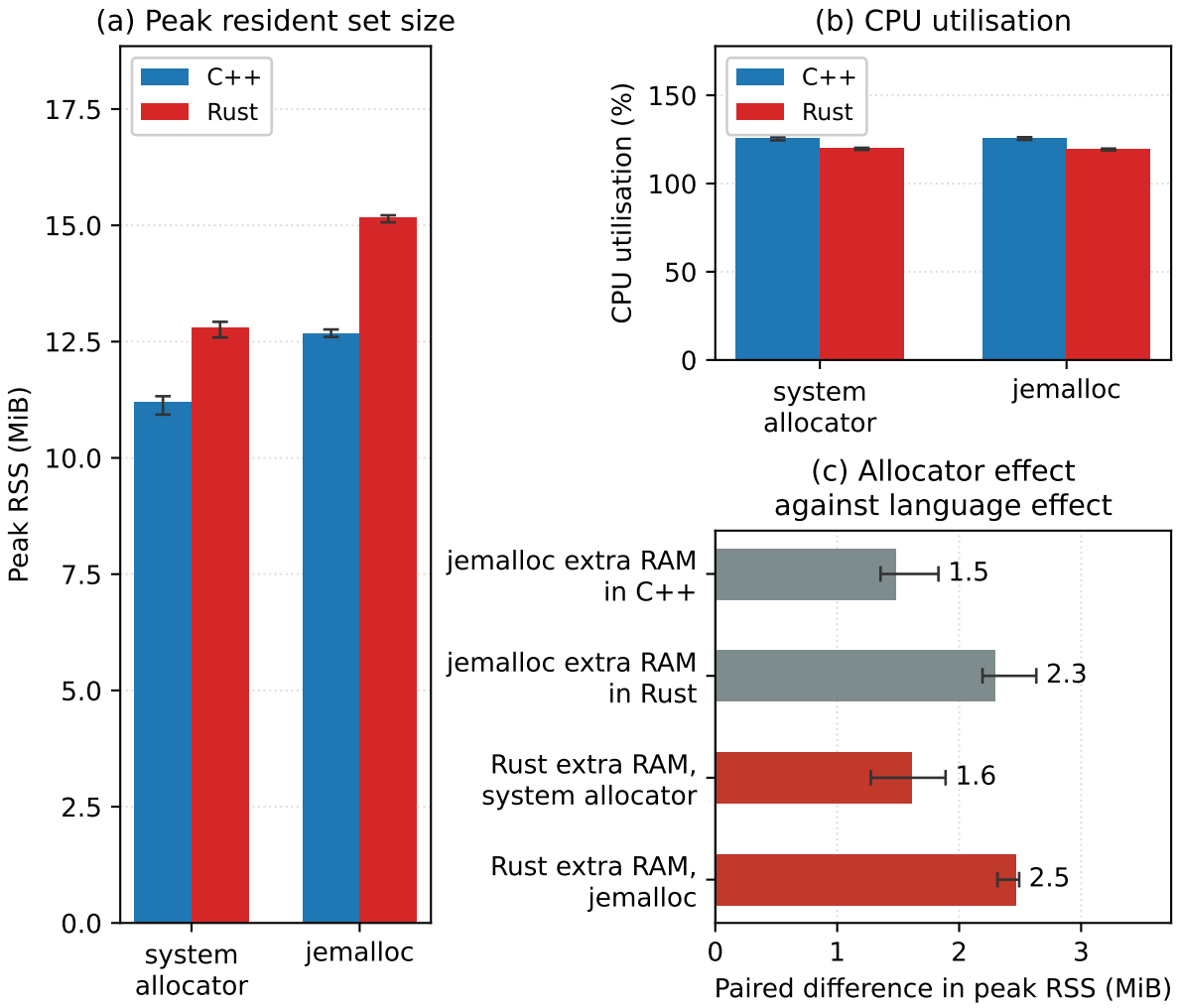


Figure 5.7: Resource use at 56 kops/s, grouped by allocator. Bars are medians over 20 repeats with bootstrap 95% intervals, and CPU in (b) is summed across threads.

5.3.4 Safety via fault injection (RQ2)

Safe code is evaluated with 14 defect classes, each written as a small program in both languages (Section 3.4) rather than injected into the service itself. Seven of these defects fall within the safety guarantees that Rust is intended to provide, while the other seven represent defects that safe Rust is not expected to prevent. The first seven defects are prevented by the Rust compiler at compile time. In C++, the same defective code is allowed to compile and run. The defect will only become visible if the program actually runs the part of the code where the defect exists. Different defects require different detection methods, so no single sanitiser can detect all of them.

Use-after-move is an important example because it is prevented by Rust even though no sanitiser reports it. A moved-from C++ object remains valid, but its state is unspecified, so

the program may continue running with the expected identity missing. There is therefore no invalid memory access for a sanitiser to detect, even though the application behaviour is incorrect (Section 3.2). Figure 5.8 shows the C++ program above the `rustc` error for its Rust version.

```
std::map<unsigned long, UeContext> store;
UeContext ctx{"244091234567890", 3};
store.emplace(1001, std::move(ctx));
// The acknowledgement is built from `ctx`, which no longer owns its identity.
std::printf("admitted UE '%s' with %d bearers\n", ctx.imsi.c_str(),
            ctx.drb_count);
```

```
error[E0382]: borrow of moved value: `ctx`
  --> faults/rust/use_after_move.rs:28:60
   |
20 |     let ctx = UeContext {
   |         --- move occurs because `ctx` has type `UeContext`, which does not implement the `Copy` trait
...
25 |     store.insert(1001, ctx); // ownership of `ctx` ends here
   |                        --- value moved here
...
28 |     println!("admitted UE '{}' with {} bearers", ctx.imsi, ctx.drb_count);
   |                                                            ^^^^^^^^^^^^^ value borrowed here after move
```

Figure 5.8: Use-after-move, the one defect inside Rust's guarantee that no sanitiser reports. The C++ code at the top, from `faults/cpp/use_after_move.cpp`, moves the context into the store, reads the moved-from variable, compiles, and under each sanitiser prints `admitted UE '' with 3 bearers` with no report. Below it, `rustc` rejects the Rust version of the same lines with error E0382, shown in part with the path shortened. A static analyser such as `clang-tidy` can flag the C++ pattern, but the study ran none.

The seven boundary classes also compile in Rust, one of them only inside `unsafe`, so their faulty behaviour can still happen when the program runs. Some can nevertheless be detected by specialised tools. In C++, LeakSanitizer, which runs as part of AddressSanitizer, reports the reference cycle. Rust's ownership model permits the same leak, so the Rust version compiles and leaks [50, 51]. UndefinedBehaviorSanitizer detects the integer overflow in C++, while Rust defines the same operation to wrap when overflow checks are disabled. ThreadSanitizer detects the lock-order inversion in 3 of 12 runs, although this result varies between runs. The tool records a lock order only when a thread that already holds one lock acquires the other. If each thread takes its first lock and then blocks on its second, neither order is ever recorded, and the tool cannot see the cycle it needs to report [52].

The double-free inside `unsafe` Rust is not a separate defect class, but the same double-free error occurring in code where Rust's normal safety checks have been bypassed. The C++ implementation contains the corresponding error, so the difference between the two implementations is only in whether the compiler is able to prevent it. Miri detects the

Rust double-free, but it cannot execute the C++ host code behind the foreign-function boundary.

Some concurrency defects cannot be found directly by the sanitiser tools. To investigate these cases, this study used two Rust tools that try many thread orders [53]. Loom tries the possible orders one by one, and Shuttle picks them at random. Both work on small test programs rather than the full application, so each defect had to be rewritten as the smallest program that still showed it. On those programs both tools report the lock-order deadlock and the lost update, and both report nothing on a fixed version of each.

RAII prevents the ordinary form of a forgotten lock in Rust. The guard releases the mutex when it is dropped, so the early return that strands the C++ lock cannot strand the Rust one. The Rust program therefore leaks its guard with `mem::forget`, a safe function that stands for any path that keeps a guard alive past the procedure that took it. Both programs then hang, and none of the three sanitisers reports the C++ hang. Figure 5.9 draws the whole list as an outcome matrix.

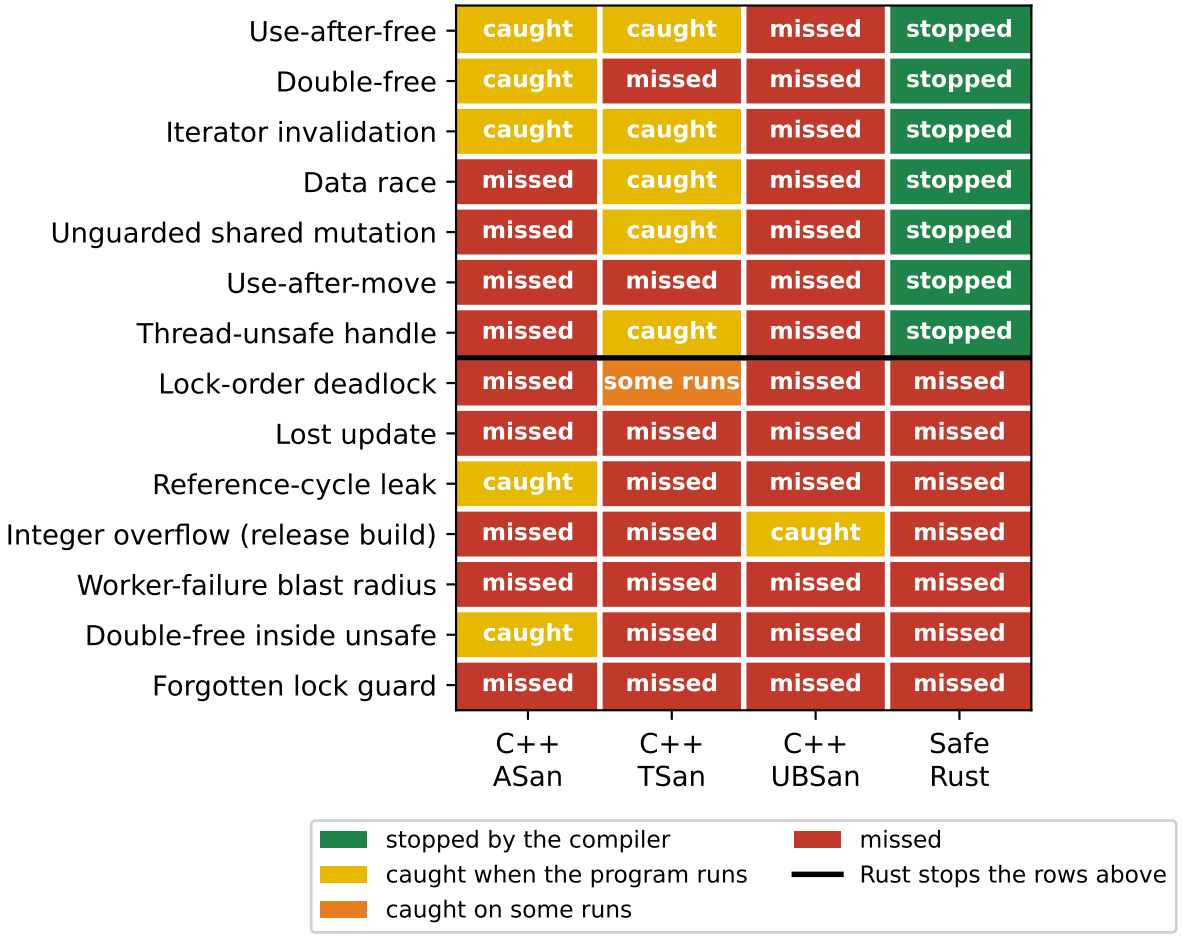


Figure 5.9: The fault-injection outcomes as a matrix, generated from the taxonomy log.

5.3.5 Migration (RQ4)

The C-ABI boundary between the C++ host and the Rust core is small. It consists of 11 C entry points implemented across 231 lines of Rust. All raw-pointer operations are isolated in 9 functions, leaving the main Rust core free of `unsafe` code.

In the paired FFI experiment, which ran on the virtual machine, the same signal call made directly and through the C-ABI cannot be told apart. Any per-call overhead the boundary adds is therefore bounded by 10.91% of the call cost in these measurements. Figure 5.10 shows the structure of this boundary.

Three practical costs of adopting Rust beyond the FFI boundary were also measured: build time, compiler cost for the project's own source, and shipped binary size. Third-party code accounts for 54% of the CPU time in a clean Rust build, which takes 8.1 s against 5.7 s for C++. For the project's own source, the Rust build needs less CPU than the whole `g++` build, 0.8 times as much, although the core library alone needs 1.5 times as much CPU in Rust as in C++. Incremental builds split the other way. After an edit inside one C++ source file the C++ build finishes in 1.2 s, against 3.6 s after the same edit in the Rust library, but an edit to the matching C++ header, which other files include, takes 5.8 s.

The shipped binaries differ by 6.8×, with Rust's the larger, mostly because the C++ build leaves its standard library to the loader and the Rust build links its own in. Linked the same way, the order depends on the policy. Statically linked, the Rust binary is the smaller, and dynamically linked, the C++ one is. Finally, the Rust core itself uses no third-party libraries and contains no `unsafe` sites (0 libraries and 0 sites). Rust's name for one library is a crate. The third-party code in the build belongs to the benchmark harness and the `jemalloc` binding, 17 and 8 libraries. The core comparison is therefore attributable to the language and build configuration rather than to dependencies or unsafe code.

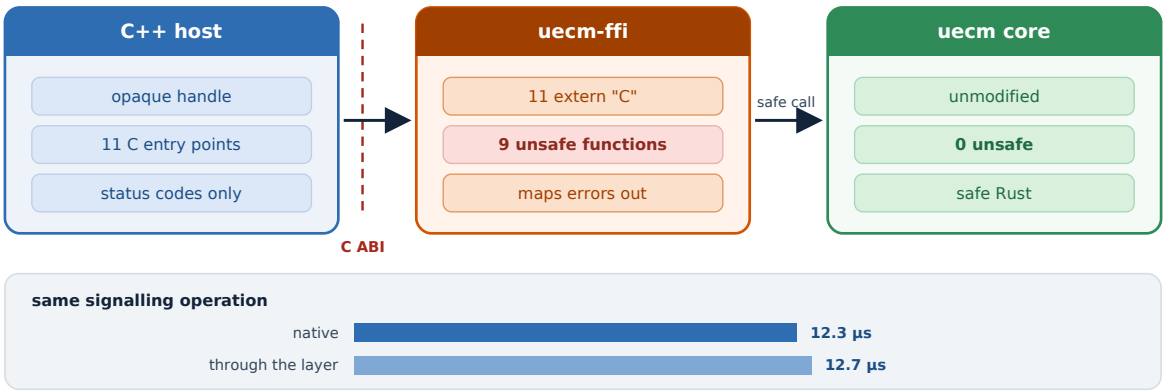


Figure 5.10: The layer where C++ calls the Rust service. The same signalling operation costs 12.3 µs directly and 12.7 µs through that layer, measured on the virtual machine of Appendix A, a paired difference bounded at 10.91% of that call. A do-nothing crossing costs 2.54 ns and bounds only itself.

6 Discussion

6.1 Summary of findings

- **RQ1 (latency).** At 56 kops/s, Rust is faster by a small but significant margin, up to 1.5 μs at the median and lower at p99 and p99.9, while the slowest calls, pooled across repeats, and the maximum favour Rust by more. That end-to-end gap lies outside the core operation, which is a small fraction of a round trip, and the clock also holds each producer's late wake-up before sending, so it cannot establish a difference in the core operation. Timed on its own on the virtual machine, the core is 1.6× faster in Rust. Past saturation the planned comparison has C++ finishing more calls, but that clock also times the sender's recording. With the recording moved after the window, Rust finishes more calls, a lead that comes from the handoff designs and not from the core operation. On further hosts with the same processor, giving Rust the C++ queue removes that lead, and giving both languages the same handoff shrinks it to 0.6% to 1.3% with the system allocator. The campaign's delay past saturation is *schedule drift* as defined in Section 3.6, so it does not compare the services. Timing the work with several requests in flight does. There the order of the total delay depends on the allocator, while the time after entering the queue is shorter for Rust under both.
- **RQ2 (defects).** The outcome follows from the fourteen-class catalogue defined in advance and does not depend on the host, apart from the intermittent ThreadSanitizer report on the lock-order deadlock. All seven classes of errors that safe Rust is designed to reject result in compile-time errors. The remaining seven compile. They fail at runtime in both Rust and C++.
- **RQ3 (resources).** Rust has higher peak RSS under both allocators. The difference is 1.6–2.5 MiB on the campaign host, 1.8–3.5 MiB on the guest, and 0.7–1.8 MiB on the second bare-metal host. Below saturation, Rust uses slightly less CPU. Past saturation the throughput order depends on where the sender records its extra delay samples, so the study gives no single throughput ranking. None of these differences is traced to the bounds checks, copies or abstraction cost that RQ3 names, which the study did not isolate.
- **RQ4 (adoption).** The per-call cost of the foreign function interface layer is bounded by 10.91% of the corresponding call. Effort is measured only through proxies: code size and complexity, the size of the `unsafe` layer, and build cost. API design and learning

effort remain literature-backed adoption costs that this study does not measure.

6.2 Discussion on the research questions

RQ1 has two answers, depending on the workload level. At 56 kops/s, the campaign detects differences in the median, p99, and p99.9 latencies, all favouring Rust, and the pooled p99.99 and the maximum, which RQ1 also names, favour Rust by more. However, the end-to-end measurements cannot isolate a difference in the core operation, because that operation is a small fraction of a round trip.

After the system reaches saturation, the campaign delay mainly reflects schedule drift rather than service performance, so it should not be used to rank the services.

Completed throughput counts calls finished during the timed window, but in the reported campaign that window also holds the sender's recording of extra delay samples. On that clock C++ completes 1.7 times as many calls as Rust. That is the planned result. With the recording moved after the window on the same host, the order reverses and Rust completes more calls with both allocators. This lead comes from the handoff between threads, where C++ spends more kernel time per call. On a further host with the same processor, replacing the Rust queue with the C++ queue removed the lead. With the same handoff in both languages, Rust stayed only 0.6% to 1.3% ahead on two further hosts with the system allocator. Most of the difference therefore comes from the queue and thread-handoff mechanism rather than from the core operation. In the C++ build that mechanism is the study's own queue, so the result compares two handoff designs as written more than two languages.

On host D, a separate experiment times each request while several requests are in flight. It covers both the signal workload and the mixed lifecycle. For each workload, the measured delay includes the wait before queue entry and the time in service. Under this measurement, the order of the total delay depends on the allocator, and the time in service is shorter for Rust under both.

The closest prior work reports Rust's p99 rising more steeply above a threshold rate and attributes it to stackless `async/await` [31]. That explanation does not apply here, because neither implementation uses an asynchronous runtime, and on the follow-up run it is C++ that falls behind first.

So a lead in finished calls does not, on its own, mean that one service has a higher maximum rate in every condition. Earlier studies would expect Rust to be slower. It checks that each index stays inside an array, a bounds check, and it can check that a

number does not grow past what its type can hold, an overflow check [54, 55]. The results show no such slowdown for the bounds checks. At the chosen load, 56 kops/s, the faster build is Rust, and Rust is the one that makes those checks. Rust's release build turns the overflow checks off. Their cost was not measured. The median at that load fits earlier evidence about a modern out-of-order processor, one that can run a check at the same time as other work, so most of the cost does not show up as extra delay [12, 34].

In safe Rust, some defects make the compiler refuse the program. Nothing is produced that can run, so that defect never happens in a run. If the compiler accepts only programs free of one kind of defect, that kind is absent from every run. A sanitiser can catch a defect only on the paths its tests actually run. Rust does not stop seven kinds of defect (Section 1.4). Those seven show where the compiler's guarantee ends. The study set out to find that edge, so they are part of the result.

Figure 6.1 shows where the guarantees provided by each approach stop. Rust provides strong guarantees about memory safety and aliasing, but it does not guarantee correct lock ordering. For example, a `Mutex` guard whose lifetime is tracked by the borrow checker does not guarantee that two threads acquire multiple locks in the same order. Static deadlock detection in Rust remains an open research problem [56].

A lost update is similarly outside the protection provided by the borrow checker and is not necessarily detected by ThreadSanitizer. When every access is correctly synchronised, there is no unsynchronised access for the tool to report. A field study of 105 real concurrency bugs found that about one third of the non-deadlock cases violated the intended ordering of operations rather than the locking rules themselves [57]. Rust therefore removes some classes of concurrency defects but does not remove defects that arise from incorrect concurrent logic.

For this reason, adopting Rust should not lead to the assumption that "memory-safe" means "concurrency-safe." Such an assumption could result in insufficient review effort. In the corpus studied by Xu et al., all but one published Rust Common Vulnerabilities and Exposures (CVE) entry related to memory safety involved `unsafe` code. The exception was a compiler bug [58].

This is why keeping the core free of `unsafe` code is important. It also explains why the spread of `unsafe` code through dependent libraries should be treated as a review concern [59].

One case shows that testing can also fail to cover a defect. A specification gap allowed a call to be sent outside the service's running window. The call passed every test, every parity vector, and all three sanitisers because the affected path was never executed. On

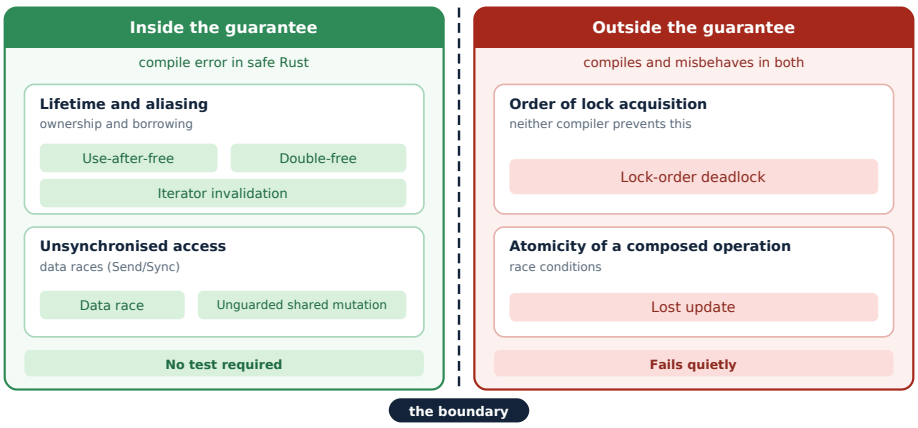


Figure 6.1: Where Rust's compile-time guarantee stops, with defect classes grouped by the mechanism that rejects them.

the Rust side, the failure became detectable because of a simple bounds check, not because of a compile-time proof. This suggests that the useful engineering principle is "checked by default," rather than "ruled out at compile time."

The root cause of the problem also belongs to the testing method. A rule without a corresponding test or enforcement mechanism leaves each implementation vulnerable to its own failure modes.

RQ3 gives different results depending on the resource being measured. The RAM result is consistent across both allocators and on a second bare-metal host with a different compiler pair, `g++ 13.3.0` and `rustc 1.98`. Peak RSS measures the maximum memory used during the entire run, so the jitter that affects p99 latency does not directly affect this metric. On both bare-metal hosts the language effect and the allocator effect are of the same few-MiB scale. On the guest system, however, the allocator effect is an order of magnitude larger. The language effect shows on every host, at a size that varies by host. What depends most on the host is how it compares with the allocator's cost.

The second bare-metal host shows where Rust's extra memory sits under `jemalloc`. On that host it lies outside what the allocator holds, mainly as a fixed cost of the process and not as more live data (Section 5.3.3). One related study reports the opposite result, with Rust using less memory and the difference increasing with concurrency [28]. These results can be reconciled if the implementations are viewed as using different concurrency runtimes rather than as direct comparisons of Rust and C++.

In that study, the C++ implementation uses Boost.Fibers, which allocates a stack for each fiber. The Rust implementation uses Tokio, where a task is represented by a small future stored on the heap. Therefore, the measured difference is partly a difference between two

concurrency runtimes.

Two further studies also found Rust using more memory than C, in a radio network management service and in a user-space device driver [29, 60]. Below saturation, Rust uses slightly less CPU. On the follow-up run past saturation, CPU per finished call differs by a factor of 1.03, with C++ the higher one, and Rust keeps a throughput lead. Neither difference can be attributed to Rust's safe-code model.

RQ3 names three costs of Rust's safety model: bounds checks, ownership-driven copies and abstraction cost. The study isolated none of them, for example by building without bounds checks. What it measures is the net cost of the Rust build. Below saturation that net cost is lower CPU and a faster core operation, against a few MiB more memory, which on the second bare-metal host sits outside the allocator. Past saturation the planned CPU comparison has Rust drawing more, on a clock that also times the sender's recording. Whatever the three named costs are in this service, they are smaller than whatever makes the Rust build faster.

RQ4 bounds what the boundary between the two languages adds. A C++ host can call an unchanged Rust core through a thin C-ABI layer, and the overhead of a real signal call is bounded by 10.91% of the total call time. RQ4 also names effort. The study measures its proxies: code size and complexity (Chapter 4), the size of the `unsafe` layer, and build cost. It does not measure the time people spend. Previous work identifies design and learning as important human costs [61, 62], and this study does not measure their magnitude.

Keeping `unsafe` code inside the interface layer limits the amount of code that requires additional review, although it does not eliminate the possibility of memory corruption. The host and the Rust component share the same address space [63, 64]. The build-size and footprint results point in the same direction: organisations concerned about these factors should examine their dependency and library budgets before attributing the cost to the compiler.

C++ keeps a clear advantage in build time: its clean build is faster, and so is its edit-and-compile cycle for changes inside a single source file (Section 5.3.5). The 6.8× size result also shows that the relative size depends on the chosen linkage policy. The ordering changes when the same policy is applied to both implementations.

6.3 Revisiting the hypotheses

Section 1.4 defined three hypotheses before any experiments were run. Revisiting them after the experiments makes clear which parts are supported and which are not.

H1 H1 is not supported. Its second clause, that any difference appears only among the slowest calls, is shown to be wrong. At 56 kops/s the paired test detects differences of up to 1.5 μs in the median and also at p99 and p99.9. Its first clause cannot be tested as written, because no equivalence margin was defined, so a small significant difference cannot show that the two are equivalent, and it cannot show that they are not. H1 also had to cover the resources of RQ3, and they move apart, with peak RSS favouring C++ and CPU favouring Rust. Holding one hypothesis for both was the weakness Section 3.3 named in advance. Past saturation lies outside H1.

H2 H2 holds for the seven in-guarantee classes this study injected. However, it does not hold under the broader interpretation suggested by its wording. The hypothesis is supported for memory safety and data-race freedom, but not for concurrency correctness in general. It also applies only to *safe* Rust code. The example involving `unsafe` code shows that the same class of defect can still occur when those guarantees are bypassed.

H3 H3 is consistent with the data but was not tested against a threshold, because, like H1, it named no margin for "small" in advance. The paired difference is bounded at 10.91% of the signalling call, on the virtual machine of Appendix A. That bound rests on the upper end of the paired interval, since a non-significant difference alone would not show that the overhead is small. The additional claim that the real cost is engineering work agrees with the structure of the integration, but this study did not measure that work.

6.4 Engineering guidelines for incremental adoption

The following guidelines translate the findings into practical recommendations. Each recommendation should be understood within the limits of the corresponding result.

- Select the first component to migrate by the simplicity of its API rather than its size. In the prototype all `unsafe` code sits in the C-ABI layer, 9 functions behind 11 entry points, so the width of the interface, not the size of the component behind it, sets how much code needs the extra review. Isolate the C ABI in a separate library and keep the core logic free of `unsafe` code, which also keeps `unsafe` code from spreading through the dependency chain [59].
- Do not reduce the budget for concurrency review. Prefer a shared-nothing design where possible, regardless of the language. In this study it is the shared-nothing design, not either compiler, that keeps shared mutable state to one atomic counter.

- Treat the learning curve as a real engineering cost and track it explicitly [61, 62].
- When benchmarking, time the component itself as well as the complete round trip, and test past the point of saturation. Audit every operation on the sender's timed path, including histogram correction and the sender's own sleep before each send. On the tested host, moving that work outside the timed window reversed which language finished more calls past saturation.

6.5 Where this study sits, and what it cannot support

The studies most closely related to this work are the five vendor-hosted theses discussed in Section 2.3. Their conclusions differ substantially from one another, and Section 6.2 reconciles the one that reports the opposite peak-RAM result.

The limitations below bound what the results support. They are grouped by what each one puts at risk.

1. **What the measures capture.** The end-to-end clock times the system design as a whole, and the core operation is a small part of it, so that clock cannot isolate a difference in the core operation. Below saturation it also includes each producer's late wake-up before sending, which both harnesses take through their own standard library, so the paired gaps there compare the two sleep paths as well as the two services. Past saturation the reported campaign's sender records extra delay samples inside the timed window, so its throughput measures that recording as well as the service. In the follow-up run that moves them, only the C++ harness stopped its timer before writing them, so Rust's throughput there is understated (Section 5.3.2). On RQ2 the C++ side is the compiler and three sanitisers with no static analyser, so its column shows the least that C++ tools can catch.
2. **What could bias the comparison.** Both versions were written under one author's direction, and much of the code in both was drafted by the same language models, as the statement on the use of AI tools records. This can introduce bias or differences in optimisation effort, and the models may write one language better than the other. A similar problem affected a previous comparison, where the result initially appeared comparable after code review but later showed the Rust version taking 85–91% longer than the C++ version, after further optimisation of both [32]. A parity test suite establishes that the implementations behave alike, not that they perform alike. The

builds also differ below the source: `g++` and `rustc` generate code differently, and the two `jemalloc` builds are different releases. Building the C++ side with `clang++`, whose code generator `rustc` shares, would separate the first. The four builds also run in the same order in every repeat, C++ first, so a steady effect of running first or last would read as a language difference. Rotating the order would separate that.

3. **How far the results carry.** The results cover one subsystem in one thread model, which a production gNB-CU-CP does not use, and every bare-metal host had the same processor, the AMD EPYC 7443P, so nothing here shows how they carry to another processor design. The defects are small standalone programs rather than faults injected into the service.
4. **What the statistics can claim.** The campaign's delay past saturation is *schedule drift*, so it does not rank the services. The experiment with several requests in flight, which does rank them, and every probe on a further host were not among the planned comparisons, so their p-values are not corrected for the number of tests.

6.6 Ethical and professional reflection

This study uses no personal data, subscriber data, or operator secrets. All traffic is generated specifically for testing. The service is written from a public specification rather than copied from production code, so the resulting artefacts can be shared publicly.

- **Claim discipline.** Avoid making claims that the data cannot support. A vendor could interpret the RQ2 result as permission to describe a migrated component as "safe." However, the seven defect classes that Rust does not prevent show why such a conclusion would be incorrect. A deadlocked control plane can cause an outage, while a silent lost-update bug can lead to incorrect mobility decisions. Rust prevents neither problem.
- **Evidence scope.** Tail-latency measurements from a single host provide a weaker basis for a production rollout than they may initially appear to provide. This is why the second bare-metal host and the earlier guest are reported beside the reported host.
- **Independence.** The work was carried out at a vendor with a stake in the language decision, the same position Section 2.3 notes for the earlier theses. The fixed public specification, the code and logs submitted with the thesis, and the boundary classes that can count against Rust are its safeguards.

7 Conclusion and Future Work

This thesis compared modern C++ and Rust on a UE Context Management service that keeps per-device state, as a gNB-CU-CP does. It asked how late the tail latency is, which defects a compiler can stop, what the safe code model costs, and what adding Rust one piece at a time costs. Both versions were built from one fixed specification and driven along the same code paths. That separates a language effect from most gaps in design, and the forced-idiom log records the rest.

Safety results. The safety mechanisms act at different stages. Safe Rust rejects all seven injected memory-safety and data-race classes before they become programs that can run. The same seven compile in C++, where a sanitiser can report a defect only if a test executes the relevant path. The other seven studied defect compile and fail in both languages. On two of them, LeakSanitizer and UndefinedBehaviorSanitizer flag the C++ version without a rewrite. For the same defects, the Rust tools did not flag anything, although Miri's leak check and a debug build's overflow checks are made for such cases. Rust therefore rules out data races and aliasing errors, but not race conditions, lock-order errors, leaks, wrapping arithmetic, or the blast radius of a failed worker. It does not stop what an `unsafe` block lets through, such as the injected double-free. Loom and Shuttle catch both concurrency cases, but only after each defect is rewritten as a small test program.

Migration and build cost. On migration, the layer where the two languages meet is small. An unchanged Rust core can be driven from C++ across that C-ABI layer. The C-ABI layer has 11 entry points in 231 lines, and its 9 raw-pointer functions contain all `unsafe` code. The core has 0 `unsafe` sites. The cost on a real signal call is bounded at 10.91% of that call. Prior work identifies API design and the learning curve as adoption costs, but this study does not measure their size. Most of the Rust build's extra cost is compiling third-party libraries, not the project's own source. The size gap comes from the linkage policy, and under a matched policy the order depends on which one is chosen. C++ leads the clean build and the edit-and-compile loop for edits that stay inside one source file.

Performance and resources. On speed the data first splits by what is being timed. The end-to-end clock starts at a request's planned send time and stops when the reply arrives. It includes the producer's late wake-up before sending, the queue, the reply, and the core call. The core work is a small fraction of that round trip, at most 0.250% in the separate timing of its parts on the virtual machine, so this clock cannot bound a compiler effect. That timing also measures the core call alone, and there the core is 1.6× faster in Rust. At the chosen load of 56 kops/s the paired test finds up to 1.5 µs in Rust's favour. At p99.99 the slowest Rust variant sits 22 µs below the fastest C++ one. Past saturation the planned comparison has C++ finishing 1.7 times as many calls, but that clock also times the sender's recording. With the recording after the timed window, Rust finishes more calls at 784 kops/s, and CPU per completed call differs by a factor of 1.03, with C++ the higher one. On a further host with the same processor, giving Rust the C++ queue removes that lead, so it comes from how calls pass between threads, not from the core work. The campaign's delay past saturation gives no language rank (Section 3.6). Timing the work with several requests in flight does rank the delay. The rank depends on the allocator, for signal and for the mixed lifecycle, and nearly all of that delay is the wait before queue entry. The time after entering the queue is shorter for Rust under both allocators. On RAM the Rust build carries a few MiB more under both allocators. It does so in the same direction and MiB scale on all three hosts timed. The allocator choice costs about as much on both bare-metal hosts, and an order of magnitude more on the guest this study used first.

Overall claim. Taken as a set, these support a narrow claim for stateful, concurrent, partitionable control-plane services of this kind. At the tested load below saturation the latency and CPU differences are small and favour Rust, against a few MiB more memory, so performance gives no reason to reject it. The choice then turns on which classes of defect a team wants to make impossible to write, and whether it will pay the design and training cost of doing so. Past saturation the order depends on how the work is timed. No single ranking holds there. The thesis also leaves three artefacts for further work: a fixed specification, a parity-tested implementation pair, and a timing harness that reaches past saturation. A split of the round trip says how much of what it times the core operation can own, and a defect list maps where the guarantee stops.

Future work.

- Vary how the kernel uses large pages for ordinary memory on one host with all else held fixed. All three hosts agree that the Rust build carries the larger footprint. They

disagree about how that sits against the cost of the allocator. The guest put it an order of magnitude higher, and neither bare-metal host does. The large-page setting is the candidate this study can name but not test, because it was not the same on each host and only the system allocator variant gets it.

- Replay anonymised production traces to check the call mix, or extend the harness to further control-plane parts or the data-heavy layer-2 and physical layer (PHY) path. A part with truly shared mutable state would work the two sides in a way a shared-nothing design does not.
- Test Rust async runtimes against the thread-per-shard model to split the runtime from the language in the tail latency result [31].
- Time what a split of the two allocators in one process costs to price what keeping a core free of `unsafe` is worth [63, 64].
- Run a longer case study that tracks defect counts and review effort while adding Rust one piece at a time. That is what would settle the human cost this study can only name.

Statement on the Use of AI Tools

Tool. Anthropic's Claude Opus 5, Claude Opus 5.5, Cursor Grok 4.6, and Cursor Grok 4.7 were used through Cursor from June 2026 until this thesis was written. University of Helsinki policy forbids naming a language model as an author. They are not cited as a source, which is the author's own choice.

What the assistance covered. The models drafted and revised the running prose, and proposed the order of the arguments. They wrote and refactored both versions of the UE Context Management service. They also wrote the load generator, the fault-injection catalogue, and the FFI prototype. The analysis behind the bootstrap intervals and the paired permutation tests is their work too, as are the figure, build and LaTeX sources.

What the models did not settle. The research problem, the four research questions, the three hypotheses, and the study's main control are the author's. That control is one frozen language-neutral specification and one concurrency model, so language is the only planned gap. The author also chose to carry defect classes from outside Rust's compile-time guarantee, which keeps RQ2 falsifiable. The author directed every measurement and chose the machines each result names. Some runs on rented machines were started by the models over a remote login, under that direction. The quantities the thesis quotes reach the page as macros rebuilt from the raw logs. The readings are the author's.

How the output was checked. The author read every model output before it entered the thesis. References were checked against Crossref, DataCite, or the publisher. A source that was never opened is listed, not used. Four passages that overstated or misplaced a source were corrected. One was a quotation that was not in the cited work. Code was checked line by line against a conformance suite and a fault catalogue. Each expected outcome was fixed before the defect was injected.

Responsibility. The author has reviewed the whole of this thesis and is responsible for all of it. Where it is wrong, the error is the author's.

Bibliography

[1] 3GPP. *NG-RAN; Architecture description*. Technical Specification (TS) 38.401. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip.

[2] 3GPP. *NR; NR and NG-RAN Overall description; Stage 2*. Technical Specification (TS) 38.300. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.300/38300-h50.zip.

[3] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia. "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges". In: *IEEE Communications Surveys & Tutorials* 25.2 (2023), pp. 1376–1411. DOI: 10.1109/COMST.2023.3239220. URL: https://doi.org/10.1109/COMST.2023.3239220.

[4] 3GPP. *NR; Radio Resource Control (RRC); Protocol specification*. Technical Specification (TS) 38.331. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), July 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.331/38331-h50.zip.

[5] 3GPP. *Study on scenarios and requirements for next generation access technologies*. Technical Report (TR) 38.913. V18.0.0, Release 18. Clause 7.4 sets the control plane latency target, for the transition from a battery-efficient state to the start of continuous data transfer, at 10 ms. 3rd Generation Partnership Project (3GPP), Apr. 2024. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.913/38913-i00.zip.

[6] J. Dean and L. A. Barroso. "The Tail at Scale". In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794. URL: https://doi.org/10.1145/2408776.2408794.

[7] L. Barroso, M. Marty, D. Patterson, and P. Ranganathan. "Attack of the Killer Microseconds". In: *Communications of the ACM* 60.4 (2017), pp. 48–54. DOI: 10.1145/3015146. URL: https://doi.org/10.1145/3015146.

[8] M. Miller. *Trends, challenge, and shifts in software vulnerability mitigation*. BlueHat IL, Microsoft Security Response Center (MSRC). Approximately 70% of Microsoft CVEs are memory-safety issues. 2019. URL: https://github.com/microsoft/MSRC-Security-Research/tree/master/presentations/2019_02_BlueHatIL.

[9] J. Vander Stoep. *Memory Safe Languages in Android 13*. Google Security Blog. Dec. 2022. URL: https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html.

[10] S. Klabnik and C. Nichols. *The Rust Programming Language*. 2nd. San Francisco, CA: No Starch Press, 2023. ISBN: 978-1-7185-0310-6. URL: https://openlibrary.org/isbn/9781718503106.

[11] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. "RustBelt: Securing the Foundations of the Rust Programming Language". In: *Proceedings of the ACM on Programming Languages* 2.POPL (2018), 66:1–66:34. DOI: 10.1145/3158154. URL: https://doi.org/10.1145/3158154.

[12] A. Balasubramanian, M. S. Baranowski, A. Burtsev, A. Panda, Z. Rakamarić, and L. Ryzhyk. "System Programming in Rust: Beyond Safety". In: *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*. 2017, pp. 156–161. DOI: 10.1145/3102980.3103006. URL: https://doi.org/10.1145/3102980.3103006.

[13] M. Polese, N. Mohamadi, S. D'Oro, L. Bonati, and T. Melodia. "Beyond Connectivity: An Open Architecture for AI-RAN Convergence in 6G". In: *IEEE Communications Magazine* 64.7 (2026), pp. 90–95. DOI: 10.1109/MCOM.001.2500438. URL: https://doi.org/10.1109/MCOM.001.2500438.

[14] ITU-R. *Framework and overall objectives of the future development of IMT for 2030 and beyond*. Recommendation ITU-R M.2160-0. Approved 13 November 2023. Defines the IMT-2030 ("6G") framework and its development timeline. International Telecommunication Union, Radiocommunication Sector, Nov. 2023. URL: https://www.itu.int/rec/R-REC-M.2160/en.

[15] AI-RAN Alliance. *AI-RAN Platform and Infrastructure Orchestrator*. AI-RAN Alliance, Working Group 2 (AI-and-RAN). Version 10-rc1, clause 2.2. Defines AI-and-RAN as one common IT infrastructure accommodating both AI and RAN workloads, which the infrastructure is required to run concurrently, and notes that the two kinds of workload may have different service-level agreements. Contributors from Ericsson, Nokia, Nvidia, Red Hat, Samsung, SoftBank, zTouch Networks and Northeastern University. Accessed August 2026. Feb. 2026. URL: https://ai-ran.org/documents/AI-RAN-Platform-Infrastructure-Orchestrator.pdf.

[16] J. Vander Stoep. *Rust in Android: Move Fast and Fix Things*. Google Security Blog. Nov. 2025. URL: https://blog.google/security/rust-in-android-move-fast-fix-things/.

[17] A. Rebert and C. Kern. *Secure by Design: Google's Perspective on Memory Safety*. Google Security Engineering, Technical Report. 2024. URL: https://research.google/pubs/secure-by-design-googles-perspective-on-memory-safety/.

[18] R. N. M. Watson, J. Baldwin, D. Chisnall, T. Chen, J. Clarke, B. Davis, N. W. Filardo, B. Gutstein, G. Jenkinson, B. Laurie, A. Mazzinghi, S. W. Moore, P. G. Neumann, H. Okhravi, A. Richardson, A. Rebert, P. Sewell, L. Tratt, M. Vijayaraghavan, H. Vincent, and K. Witaszczyk. "It Is Time to Standardize Principles and Practices for Software Memory Safety". In: *Communications of the ACM* 68.2 (2025), pp. 40–45. DOI: 10.1145/3708553. URL: https://doi.org/10.1145/3708553.

[19] Cybersecurity and Infrastructure Security Agency. *The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously*. Cybersecurity Information Sheet. CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023. URL: https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps.

[20] Office of the National Cyber Director. *Back to the Building Blocks: A Path Toward Secure and Measurable Software*. Technical Report. Washington, DC: The White House, Office of the National Cyber Director (ONCD), Feb. 2024. URL: https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf.

[21] European Parliament and Council of the European Union. *Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)*. Official Journal of the European Union, OJ L, 2024/2847. Nov. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj.

[22] Dell'Oro Group. *RAN Mostly Stable in 3Q 2025*. Dell'Oro Group, RAN Quarterly Report summary. The five largest RAN suppliers by worldwide revenue (Huawei, Ericsson, Nokia, ZTE, Samsung) accounted for 96% of the 1Q25–3Q25 RAN market, up from 95% in 2024. Accessed August 2026. Nov. 2025. URL: https://www.delloro.com/news/ran-mostly-stable-in-3q-2025/.

[23] Dell'Oro Group. *RAN Market Stabilized in 2025*. Dell'Oro Group, RAN Quarterly Report summary. Full-year 2025 preliminary findings: supplier rankings unchanged, and overall market concentration measured by the Herfindahl–Hirschman Index

reached a ten-year high. Accessed August 2026. Feb. 2026. URL: https://www.delloro.com/news/ran-market-stabilized-in-2025/.

[24] J. Almodovar and A. Sultan. *The SA1 road to 6G*. 3GPP Highlights, Issue 09. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work. ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026. Dec. 2024. URL: https://www.3gpp.org/news-events/3gpp-news/sa1-6g-road.

[25] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang. "Understanding Memory and Thread Safety Practices and Issues in Real-World Rust Programs". In: *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2020, pp. 763–779. DOI: 10.1145/3385412.3386036. URL: https://doi.org/10.1145/3385412.3386036.

[26] 3GPP. *Service requirements for the 5G system; Stage 1*. Technical Specification (TS) 22.261. V18.6.0, Release 18. 3rd Generation Partnership Project (3GPP), Mar. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-i60.zip.

[27] 3GPP. *NG-RAN; E1 Application Protocol (E1AP)*. Technical Specification (TS) 38.463. V17.0.0, Release 17. 3rd Generation Partnership Project (3GPP), Apr. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.463/38463-h00.zip.

[28] R. Karlbäck and A. Orö. "Holistic View on Alternative Programming Languages for Radio Access Network Applications in Cloud and Embedded Deployments". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2021. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307.

[29] V. Olsson. "Rust Programming Language as an Alternative to C for RAN Management Applications". Carried out at Ericsson AB. Master's thesis. KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-325822.

[30] J. Olsson and I. Stenström. "Rust Async for Thread-per-Core Packet Processing". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-197029.

[31] A. Arvidsson and M. Kaller. "Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2025. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-214746.

[32] C. Emberg and A. Wadsten. "Attainable Safety and Long-Term Resource-Efficiency Using Rust: Evaluating the Viability of Rust in Terms of Development Effort and Performance for the Decompilation of ASN.1 Messages". Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-195552.

[33] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva. "Energy Efficiency Across Programming Languages: How Do Energy, Time, and Memory Relate?" In: *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*. 2017, pp. 256–267. DOI: 10.1145/3136014.3136031. URL: https://doi.org/10.1145/3136014.3136031.

[34] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. "The Case for Writing Network Drivers in High-Level Programming Languages". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–13. DOI: 10.1109/ANCS.2019.8901892. URL: https://doi.org/10.1109/ANCS.2019.8901892.

[35] F. Suchert and J. Castrillon. "STAMP-Rust: Language and Performance Comparison to C on Transactional Benchmarks". In: *Benchmarking, Measuring, and Optimizing (Bench 2022)*. Vol. 13852. Lecture Notes in Computer Science. Springer, 2023, pp. 160–175. DOI: 10.1007/978-3-031-31180-2_10. URL: https://doi.org/10.1007/978-3-031-31180-2_10.

[36] R. Gjomemo, P. H. Phung, E. Ballou, K. S. Namjoshi, V. N. Venkatakrishnan, and L. Zuck. "Leveraging Static Analysis Tools for Improving Usability of Memory Error Sanitization Compilers". In: *Proceedings of the 2016 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. Two of the six authors are affiliated with Nokia. 2016, pp. 323–334. DOI: 10.1109/QRS.2016.44. URL: https://doi.org/10.1109/QRS.2016.44.

[37] B. Schroeder, A. Wierman, and M. Harchol-Balter. "Open Versus Closed: A Cautionary Tale". In: *Proceedings of the 3rd USENIX Symposium on Networked Systems*

Design and Implementation (NSDI '06). 2006. URL: https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale.

[38] R. Natella, D. Cotroneo, and H. S. Madeira. "Assessing Dependability with Software Fault Injection: A Survey". In: *ACM Computing Surveys* 48.3 (2016), 44:1–44:55. DOI: 10.1145/2841425. URL: https://doi.org/10.1145/2841425.

[39] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. "Detecting Cross-Language Memory Management Issues in Rust". In: *Computer Security – ESORICS 2022*. 2022, pp. 680–700. DOI: 10.1007/978-3-031-17143-7_33. URL: https://doi.org/10.1007/978-3-031-17143-7_33.

[40] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993. ISBN: 978-0-412-04231-7.

[41] S. Holm. "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6 (1979), pp. 65–70. URL: https://www.jstor.org/stable/4615733.

[42] P. Enberg, A. Rao, and S. Tarkoma. "The Impact of Thread-per-Core Architecture on Application Tail Latency". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–8. DOI: 10.1109/ANCS.2019.8901874. URL: https://doi.org/10.1109/ANCS.2019.8901874.

[43] G. Tene. *HdrHistogram: A High Dynamic Range Histogram*. Project documentation and reference implementation. Section "Corrected vs. raw value recording calls" documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026. URL: https://github.com/HdrHistogram/HdrHistogram.

[44] G. L. Steele Jr., D. Lea, and C. H. Flood. "Fast Splittable Pseudorandom Number Generators". In: *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*. 2014, pp. 453–472. DOI: 10.1145/2660193.2660195. URL: https://doi.org/10.1145/2660193.2660195.

[45] L. C. Noll, K.-P. Vo, D. E. Eastlake 3rd, and T. Hansen. *The FNV Non-Cryptographic Hash Algorithm*. Request for Comments (RFC) 9923. Informational. Specifies the FNV-1a hash used inside each shard. Internet Engineering Task Force (IETF), Feb. 2026. DOI: 10.17487/RFC9923. URL: https://www.rfc-editor.org/rfc/rfc9923.

[46] T. Yin. *Lizard: An extensible cyclomatic complexity analyzer for many programming languages*. Version 1.23.0. Accessed August 2026. 2026. URL: https://github.com/terryyin/lizard.

[47] The Linux man-pages project. *PR_SET_TIMERSLACK(2const): Set the Current Timer Slack Value for the Calling Thread*. Linux man-pages 6.19, 2026-02-08. Accessed October 2026. 2026. URL: https://man7.org/linux/man-pages/man2/PR_SET_TIMERSLACK.2const.html.

[48] A. Georges, D. Buytaert, and L. Eeckhout. "Statistically Rigorous Java Performance Evaluation". In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. 2007, pp. 57–76. DOI: 10.1145/1297027.1297033. URL: https://doi.org/10.1145/1297027.1297033.

[49] X. Zhou, V. Leis, X. Yu, and M. Stonebraker. "OLTP Through the Looking Glass 16 Years Later: Communication Is the New Bottleneck". In: *15th Annual Conference on Innovative Data Systems Research (CIDR 2025)*. 2025. URL: https://www.vldb.org/cidrdb/papers/2025/p17-zhou.pdf.

[50] J. Hughes and L. Tratt. "Garbage Collection for Rust: The Finalizer Frontier". In: *Proceedings of the ACM on Programming Languages* 9.OOPSLA2 (2025), pp. 3588–3614. DOI: 10.1145/3763179. URL: https://doi.org/10.1145/3763179.

[51] M. Cui, H. Xu, H. Tian, and Y. Zhou. "rCanary: Detecting Memory Leaks Across Semi-Automated Memory Management Boundary in Rust". In: *IEEE Transactions on Software Engineering* 50.9 (2024), pp. 2472–2484. DOI: 10.1109/TSE.2024.3443624. URL: https://doi.org/10.1109/TSE.2024.3443624.

[52] The Sanitizers Project. *ThreadSanitizer deadlock detector*. Project documentation. Accessed August 2026. 2026. URL: https://github.com/google/sanitizers/wiki/ThreadSanitizerDeadlockDetector.

[53] J. Bornholt, R. Joshi, V. Astrauskas, B. Cully, B. Kragl, S. Markle, K. Sauri, D. Schleit, G. Slatton, S. Tasiran, J. Van Geffen, and A. Warfield. "Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3". In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 836–850. DOI: 10.1145/3477132.3483540. URL: https://doi.org/10.1145/3477132.3483540.

[54] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu. "Towards Understanding the Runtime Performance of Rust". In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2022, 140:1–140:6. DOI: 10.1145/3551349.3559494. URL: https://doi.org/10.1145/3551349.3559494.

[55] S. Harris, S. Cooksey, M. Vollmer, and M. Batty. "Rust for Morello: Always-On Memory Safety, Even in Unsafe Code (Experience Paper)". In: *Proceedings of the 37th European Conference on Object-Oriented Programming (ECOOP)*. Vol. 263. Leibniz International Proceedings in Informatics (LIPIcs). 2023, 39:1–39:27. DOI: 10.4230/LIPIcs.ECOOP.2023.39. URL: https://doi.org/10.4230/LIPIcs.ECOOP.2023.39.

[56] Y. Zhang, K. Zhang, G. Liu, Y. Cai, and S. Qin. "Two Birds One Stone: Effective Static Detection of Resource and Communication Deadlocks in Rust Programs". In: *Automated Software Engineering* 33.2, 65 (2026). Extends the preprint arXiv:2401.01114. DOI: 10.1007/s10515-026-00614-z. URL: https://doi.org/10.1007/s10515-026-00614-z.

[57] S. Lu, S. Park, E. Seo, and Y. Zhou. "Learning from Mistakes: A Comprehensive Study on Real World Concurrency Bug Characteristics". In: *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2008, pp. 329–339. DOI: 10.1145/1346281.1346323. URL: https://doi.org/10.1145/1346281.1346323.

[58] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M. R. Lyu. "Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs". In: *ACM Transactions on Software Engineering and Methodology* 31.1 (2022), 3:1–3:25. DOI: 10.1145/3466642. URL: https://doi.org/10.1145/3466642.

[59] A. N. Evans, B. Campbell, and M. L. Soffa. "Is Rust Used Safely by Software Developers?" In: *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. 2020, pp. 246–257. DOI: 10.1145/3377811.3380413. URL: https://doi.org/10.1145/3377811.3380413.

[60] A. Sundvisson and S. Hofer. "User-Space Device Drivers on Embedded Linux: A Comparative Case Study of C and Rust". Bachelor's thesis. Blekinge Institute of Technology, Department of Software Engineering, 2026. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:bth-29775.

[61] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. "Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study". In: *Proceedings of the 17th Symposium on Usable Privacy and Security*

(SOUPS). USENIX Association, 2021, pp. 597–616. URL: https://www.usenix.org/conference/soups2021/presentation/fulton.

[62] W. Crichton and S. Krishnamurthi. "Profiling Programming Language Learning". In: *Proceedings of the ACM on Programming Languages* 8.OOPSLA1 (2024), pp. 29–54. DOI: 10.1145/3649812. URL: https://doi.org/10.1145/3649812.

[63] E. Rivera, S. Mergendahl, H. Shrobe, H. Okhravi, and N. Burow. "Keeping Safe Rust Safe with Galeed". In: *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*. 2021, pp. 824–836. DOI: 10.1145/3485832.3485903. URL: https://doi.org/10.1145/3485832.3485903.

[64] P. Kirth, M. Dickerson, S. Crane, P. Larsen, A. Dabrowski, D. Gens, Y. Na, S. Volckaert, and M. Franz. "PKRU-Safe: Automatically Locking Down the Heap Between Safe and Unsafe Languages". In: *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*. 2022, pp. 132–148. DOI: 10.1145/3492321.3519582. URL: https://doi.org/10.1145/3492321.3519582.

Appendix A Host and campaign configuration

Table A.1 itemises the three machines the campaigns ran on. The first is the bare-metal host of Section 5.1 that every performance and resource table of the main sweep is drawn from. The second is the virtualised host it replaced. The third is a second bare-metal host with a different design. Section 5.1 compares that third host with the reported host. The resident-set decomposition belongs to the second bare-metal host and is reported in Section 5.3.3. The fault catalogue was compiled and run on the development machine. Of the adoption-cost axes only build time is host-sensitive, so its seconds describe the development machine and only its ratios carry to another one, while the footprint and dependency figures are deterministic. The three small paired experiments predate the move to bare metal. The split of a round trip and the FFI comparison ran on the virtual machine, and the queue timing on a host that was not recorded. None carries an environment record, so each supports a ratio between two variants measured alternately in one session. Section 5.1 points here for the field-by-field record of what each campaign captured. Every column reports what was measured, and the protocol they are read against is published as `bench/TESTBED.md`. In Table A.1, the reported sweep and the virtualised host share the design and differ in the machine, the kernel, the core isolation and pinning, the large-page setting and the toolchain. The virtual machine had `g++ 15.1.1` installed directly, before the container image existed, and the reported sweep took `g++ 15.2.1` from that image. The second bare-metal host differs further in shard count, producer count, load grid, repeat count and compiler, and it ran without `nohz_full`. Table A.2 gathers the configuration of the service, the workload and the campaign, and Table A.3 lists the ten planned comparisons below saturation. Table 5.2 in Section 5.3.2 gives the four past it.

Table A.1: The three machines the measurements ran on. A *Short of protocol* mark is a place where that machine missed the written test plan.

Aspect	Reported sweep	Virtualised host, same design	Second bare-metal host, different design
Host	AMD EPYC 7443P, 24 cores, 1 socket	Intel(R) Xeon(R) CPU E5-2680 v4 guest, 16 vCPUs	AMD EPYC 7443P, 24 cores, 1 socket
Operating system	Ubuntu 24.04.4 LTS. Image Rocky Linux 8.10 (Green Obsidian)	Rocky Linux 8.10 (Green Obsidian)	Ubuntu 24.04.4 LTS
Kernel	6.17.0-23-generic	7.0.2-1.el8.elrepo.x86_64	6.17.0-23-generic
Hypervisor	none. Clock setting performance, boost off, verified on the machine	kvm. Frequency and turbo host-controlled, unobservable from inside	none. Clock setting performance, verified on the machine
CPU steal	None: no hypervisor to lose a timeslice to	Not characterised. The capture's single one-second sample read 0, and no steal probe was run on this host	0 of 60 intervals on 16 pinned cores
Isolation	`nohz_full` 2-17. `isolcpus` none, omitted on purpose (§5.1)	`isolcpus` none, `nohz_full` none	`isolcpus` none, `nohz_full` none. *Short of protocol*
Large pages	madvise	always, and only the system-allocator variant inherits it	madvise
Toolchain	`g++ 15.2.1`, `rustc 1.97.1`, pinned in a container image	`g++ 15.1.1`, `rustc 1.97.1`	`g++ 13.3.0`, `rustc 1.98`, so a compiler change is tested too
Pinning	Cores 2–17, 7 shards, 7 producers	Cores 2–15, 7 shards, 7 producers	Cores 2–17, 8 shards, 8 producers
SMT	off, verified	Absent from the topology exposed to the guest, and decided on the host	off, verified
Repeats	20 over 14 loads, 1120 runs	20 over 14 loads, 1120 runs	15 over 8 loads, 480 runs. *Short of protocol*
Instruments	Pooled HDR sidecars, jitter, allocator accounting	None available in the guest	Counters at two scopes, HDR sidecars, allocator accounting
Adequate for	The pre-committed campaign-clock results. Not overload service latency	Median, CPU, overload, and peak RSS on the system allocator. Not p99, p99.9 or p99.99	Confirming campaign-clock directions survive a change of design

Table A.2: Settings of the service, the workload, and the campaign. Both languages use the same settings.

Parameter	Value	Note
Service under test, identical in both languages		
Shards, and worker threads	7	one thread owns one shard. Identical in both languages, which is what the fairness control requires (§3.5)
Queue bound per shard	1 024 requests	bounded and blocking, so overload is observable
Initial map capacity	4 096 entries	reserved so steady state does not rehash
Bearers per context	at most 8	
Signalling payload	64 bytes	
Context ceiling per shard	200 000	
Shard selection	SplitMix64, then modulo	pinned: diffuses sequential identifiers
Intra-shard hash	FNV-1a, 64-bit	pinned on both sides and checked bit-for-bit
Workload		
Workload	mixed	walks the full RAN lifecycle. The signal workload runs only in the experiment with several requests in flight
Active UEs	5 000	
Offered-load levels	14	up to 784 kops/s
Representative load	56 kops/s	fixed before the campaign (§5.2)
Plateau of the slowest variant	392 kops/s achieved	all four variants track through 392 kops/s. Rust with `jemalloc` also tracks at 448 kops/s. Past saturation this campaign records the extra delay samples inside the timed window (Section 5.3.2)
Load model	fixed send schedule, one request in flight per producer	latency timed from the planned send time, producers at ordinary priority
Campaign		
Build variants	4	two languages × two allocators
Repeats per sweep point	20	variants interleaved within each repeat, in the same order every time
Warm-up	1 s per run	sent at the offered rate and not recorded
Measured runs	1120	
Latency capture	HDR histogram, $[1\,\text{ns}, 3\,600\,\text{s}]$, 3 s.f.	same algorithm and configuration on both sides
Summary statistic	median with bootstrap 95% confidence interval	intervals reported in full, not as half-widths
Language comparison	exact paired permutation test	14 tests, Holm-corrected
Attainable p floors	0.0000019 / 0.000027	uncorrected / after correction

Table A.3: The ten planned comparisons below saturation, Rust minus C++. A negative number means Rust is lower. The number above a bracket is the median difference, and the bracket is the 95% interval. The lower block holds the high percentiles, which a few slow calls can move.

Metric	Allocator	Rust minus C++	p	Holm p
Robust to interference on single operations				
p50 latency (µs)	system	-1.2 [-1.3, -1.1]	0.0000019	0.000027
p50 latency (µs)	jemalloc	-1.5 [-1.6, -1.5]	0.0000019	0.000027
Peak RSS (MiB)	system	+1.6 [+1.3, +1.9]	0.0000019	0.000027
Peak RSS (MiB)	jemalloc	+2.5 [+2.3, +2.5]	0.0000019	0.000027
CPU utilisation (%)	system	-6.0 [-7.2, -4.6]	0.0000019	0.000027
CPU utilisation (%)	jemalloc	-6.2 [-7.3, -5.4]	0.0000019	0.000027
Determined by it: high percentiles				
p99 latency (µs)	system	-2 [-2, -1]	0.0000019	0.000027
p99 latency (µs)	jemalloc	-1 [-1, -1]	0.0000019	0.000027
p99.9 latency (µs)	system	-4 [-4, -4]	0.0000019	0.000027
p99.9 latency (µs)	jemalloc	-3 [-3, -2]	0.0000019	0.000027

What each campaign recorded. Each of the 1120 runs of the factorial design in Table A.2 contributes one newline-delimited JSON record. Nothing is aggregated at capture time. A record carries the configuration (language, allocator, workload, shard and producer counts, target rate), the outcome (operations completed, errors, wall-clock duration, achieved throughput), and the latency distribution from the HDR histogram (mean, standard deviation, the quartiles, p50, p90, p99, p99.9, p99.99 and the maximum, in nanoseconds). It also carries a shorter block for inter-operation latency variation (mean, standard deviation, p50, p99, p99.9, the maximum and a sample count), the resource counters from `getrusage` (peak resident set size, user and system CPU seconds, CPU utilisation), and the per-operation-kind counts. A `jemalloc` variant adds that allocator's own accounting. A system-allocator variant stores those fields as unavailable rather than as zero, on the same principle as the turbo state. The kind counts are what makes a divergence between the languages show up as a parity failure (Chapter 4) instead of a performance difference. Each run also persists its complete HDR bucket contents to a sidecar, so the repeats can be pooled before the time of the slowest calls is read (Section 3.6). The second bare-metal campaign also has a separate file of `perf` counters at two scopes, the end-to-end path and the operation on its own.

The three small paired experiments have a different shape. For each of 15 interleaved repeats the decomposition records the batch-mean cost in both languages, once for the operation called directly and once through the threaded service. A third dataset of the same shape over 20 repeats records sustained throughput through the bounded queue and completion primitives alone. With 7 producers, Rust sustained 59 kops/s and C++ 55, which the test cannot separate from no difference ($p = 0.226$). The FFI comparison of Section 5.3.5 has that structure over 15 repeats. All three predate the environment record, which is why each is read as a ratio between its two variants.

Build time is the one RQ4 axis that is a timing measurement. It is recorded over 15 interleaved repeats at 14 parallel jobs on the development machine, alternating the toolchains so the ratio survives a change of machine even though the seconds do not. The static footprint and the resolved dependency graph are read from the linked binaries and the manifests.

The fault catalogue is categorical. Each of the 14 cases yields four tool logs (AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer on the C++ program, and `rustc` on the safe-Rust counterpart) and one taxonomy row carrying the three sanitiser outcomes, the detection stage, the compiler result, and the expectation registered before the study. The lock-order deadlock and the lost update also yield a log from each of the two Rust tools, Loom and Shuttle, and the case inside `unsafe` a log from Miri, each beside a

fixed version of the program. The outcome matrix in Section 5.3.4 is generated from the taxonomy rows.

Every load sweep writes the environment record of Section 3.7 beside its results, including the fields a given host does not expose. Such fields, the turbo state among them, are stored as unavailable rather than left out, so what a host could not report is a property of the dataset. The virtualised campaign shares the sweep's design and predates the later instruments. Its records carry the configuration, the percentiles, the resource counters and the kind counts, without histogram sidecars, jitter or allocator accounting. Its wide intervals are the measurement-validity argument of Section 5.1, which is why the campaign is kept. The second bare-metal campaign carries the same record shape as the reported sweep, including the allocator accounting that lets Section 5.3.3 decompose the resident set. It runs a different shard count, load grid, repeat count and toolchain. Each campaign has its own generator and macro prefix.

Two follow-up runs repeat the campaign's grid of 14 loads, two languages and two allocators. The first writes no extra delay samples at all and ran on an earlier host, so its percentiles miss slow calls it should have counted and are never quoted. The second writes them after the timed window, on the campaign host: the processor is AMD EPYC 7443P, the pin is 2–17, and `nohz_full` is 2-17, matching the campaign's AMD EPYC 7443P and `2-17`. Its overload percentiles are *schedule drift* as defined in Section 3.6. At the top load Rust is ahead on both allocators, at $p < 0.001$ on the system allocator and $p < 0.001$ under `jemalloc`, the reverse of the planned comparison. CPU per completed operation differs by a factor of 1.03, with C++ the higher one.

The tables that follow provide supporting measurements for the main text.

Table A.4: How much the delay changes from one call to the next, at the chosen load. It is measured inside one producer, in microseconds. The number above a bracket is the median, and the bracket is the 95% interval. It is not one of the planned comparisons.

Variant	p50	p99	p99.9	max
C++ (system alloc)	1.4	22.3	43.6	154.0
	[1.3, 1.5]	[18.6, 33.4]	[39.6, 47.9]	[148.9, 167.0]
C++ (jemalloc)	1.6	22.5	43.7	158.7
	[1.5, 1.6]	[14.3, 31.9]	[42.4, 46.6]	[148.2, 170.0]
Rust (system alloc)	1.5	26.9	40.7	56.9
	[1.3, 1.7]	[20.9, 33.7]	[38.7, 45.1]	[53.4, 107.5]
Rust (jemalloc)	1.6	20.9	44.9	63.4
	[1.6, 1.7]	[15.9, 26.1]	[42.1, 46.6]	[55.7, 108.9]

The allocation profile on the second bare-metal host was taken to name the call sites

Table A.5: Where the extra memory sits on the second bare-metal host, in MiB. These figures are medians over the steady loads. The last column is Rust minus C++ on the same run, so the rows need not add up to it.

Component	C++	Rust	Difference
Live allocation (`allocated`)	0.78	0.62	-0.16
In active pages (`active`)	1.43	0.79	-0.64
Arena bookkeeping (`metadata`)	4.19	6.26	+2.07
Resident in allocator mappings (`resident`)	11.02	9.10	-1.92
Outside those mappings (derived)	4.04	7.86	+3.82
Peak resident set (`getrusage`)	15.08	16.82	+1.74

behind the resident-set difference of Section 5.3.3. Two departures from the campaign explain why it cannot. The first profile sent requests back to back without a schedule, at 455 kops/s, inside the bracket of 448–512 kops/s where that host's throughput stops rising. Repeating it at a swept load below saturation shrank the discrepancy by most of its size and did not remove it. What remains is that profiling requires an allocator built to sample every allocation and record a backtrace, so the variant under profile is not the variant the campaign measured. Each symbolised profile puts almost all of its flat bytes on a frame that allocates nothing. In the C++ variant 98.4% sit on a deallocation symbol. In the Rust variant 100.0% sit on the allocator's own backtrace routine. Attributing the footprint would need a profile taken at a swept load, on a build whose peak RSS reproduces the campaign's ordering.

Appendix B Overload probes in detail

The probes of Section 5.3.2 ran on four rented bare-metal hosts, listed in Table B.1. Each is a single-socket AMD EPYC 7443P with SMT off, the performance governor and `nohz_full` on the measured cores, without `isolcpus`. Each ran the campaign's container image with the source as it stood on its date, so hosts B and C ran a service with the asynchronous interface added on 7 October, which the campaign's build did not have. None of them is the reported campaign host, and no probe is one of the planned comparisons.

Table B.1: The other machines used for the follow-up runs.

Host	Where and when	What ran there
A	Singapore, 30 September 2026	The map swap, the queue swap, the ring buffer, and thread switches per call
B	Lithuania, 19 October 2026	The same-handoff probe, kernel calls per request, and the timer test
C	Chicago, 20 October 2026	Time per part of a call, and a repeat of the same-handoff probe
D	Chicago, 6 October 2026	Several requests in flight

The same-handoff probe runs the real `ShardCore` in one program per language, behind one queue and one reply slot per sender. Both versions make the same Linux `futex` system calls, and they wake a thread only after it has marked itself as sleeping. The same program can call the reported service instead, so within one binary only the handoff changes. Senders send back to back without a timetable, and no histogram is kept.

On the system allocator Rust stayed ahead with the same handoff in 17 of 20 paired repeats on host B and in 16 of 20 on host C. Under `jemalloc` on host B the gap fell from 11.8% to 1.4%, which the test cannot separate from no difference ($p = 0.174$). With the same handoff both languages make 3.19 system calls per request. Rust spends 0.12 µs less user time per call, out of a 12.6 µs round trip.

On host A the C++ lock adds kernel time to each handoff at the highest load. All four builds of the queue swap switch threads 2.8 times per call at 56 kops/s, below saturation, so at that load the lock causes no extra thread sleeps. No switch count was taken at the highest load.

The timer test on host B set the cap on the deferred samples to zero. That removes only those samples. Rust's throughput at the highest load rose by 2.4% and was higher in 9 of

10 paired repeats, while C++ did not change. After the Rust harness was changed to stop its timer before writing them, the same test no longer moved Rust.

On host C the probe stamped each call as the sender handed it over, as the worker started and finished it, and as the reply arrived, all on one clock. The worker's two stamps are the reported services' own, so every build pays the same two extra clock reads. The parts add up to each run's time per call.